CS-4407 | TEST-2 EXAM NOTES

Information System Design

Unit III & Unit IV — Complete Notes
with simple explanations, examples, diagrams and probable questions

Unit III: Requirements Engineering and Software Design
Unit IV: Object-Oriented Analysis and Design (UML)

Test format: 4 questions × 5 marks = 20 marks
Prepared from your syllabus and the class slides (L5, L8, L9, L10) and mam's Examination System DFD

How to use these notes

Read this first

EXAM STRATEGY

You get **4 questions for 20 marks**, so about 5 marks each. In 5 marks, examiners expect: a short definition, 4–5 clear points, and **one neat diagram or example**. Diagram questions (DFD, use case, class, sequence, state, activity) are very likely, because mam has already shared some.

Unit III — Requirements Engineering and Software Design

3.1 Requirements and types of requirements

3.2 Requirements engineering process and Elicitation

3.3 Requirements analysis: Structured (DFD, mam's Examination System) vs Object-oriented, Prototyping

3.4 Requirements specification (SRS, decision table, PDL, FSM and more)

3.5 Requirements validation, management and tools

3.6 Software design: fundamentals, design process, modular design, component-level design

3.7 Structured vs Object-oriented design, Refactoring

3.8 Probable questions for Unit III

Unit IV — Object-Oriented Analysis and Design

4.1 Object-oriented basics: object, class, encapsulation, inheritance, polymorphism, relationships

4.2 UML: history, building blocks, rules, mechanisms, 4+1 views

4.3 Use case diagram, use case scenario and use case relationships

4.4 Class diagram and Object diagram

4.5 Sequence diagram and Collaboration diagram

4.6 State-chart diagram and Activity diagram

4.7 Component diagram and Deployment diagram

4.8 Generalization, Domain model refinement, Architecture, Packaging, Large-scale case study

4.9 Probable UML diagram questions with solutions

4.10 Quick revision: differences and one-page cheat sheet

UNIT III

Requirements Engineering • Software Design

3.1 What is a requirement?

DEFINITION

A **requirement** is a description of a feature or service that the system must give, within some limits (constraints). Requirements collected from the customer are written in a formal document called the **SRS (Software Requirements Specification)**.

REQUIREMENTS ENGINEERING (RE)

RE is the process of **gathering, analysing, documenting, validating and managing** requirements. Its main goal is to understand what the customer really wants and to organise it clearly in the SRS.

Why is RE important? It decides *what to build*. Wrong or unclear requirements are the number one reason why projects fail, and fixing them later is very costly.

Types of requirements

Type	Meaning	Example (ATM / Library)
Business requirements	Why the project is needed: goals and expected business benefits. Written by management / business analyst.	"Reduce bank queue time by 50%."
User requirements	High-level statements in natural language from users and stakeholders. May be incomplete or ambiguous.	"Member should be able to issue a book easily."
System requirements	Detailed technical description of what the system will do. Acts like a *contract* between client and developer.	"System shall allow max 3 books per member and generate a due date of 14 days."
Functional (FR)	**What** the system must do: functions, transactions, authentication, reports, interfaces.	Login, issue book, withdraw cash, print receipt.
Non-functional (NFR)	**How well** the system must work: quality and constraints such as speed, security, usability, reliability, scalability, availability, maintainability.	"Search result must come in 2 seconds." "System must be available 24x7."

EXAMPLE — ETRANSQ (FROM YOUR SLIDES)

EtransQ is a web-based system where **suppliers, transporters and agents** share a common platform.
Business: give a common platform, easy search for tenders and offers, build trust.
Functional: registration, association, quick search, tendering, offering, testimonial.
Non-functional: availability, recoverability, security, privacy, usability, maintainability, reusability, robustness.

3.2 Requirements Engineering process

RE has two main parts: **Requirement Development** and **Requirement Management**.

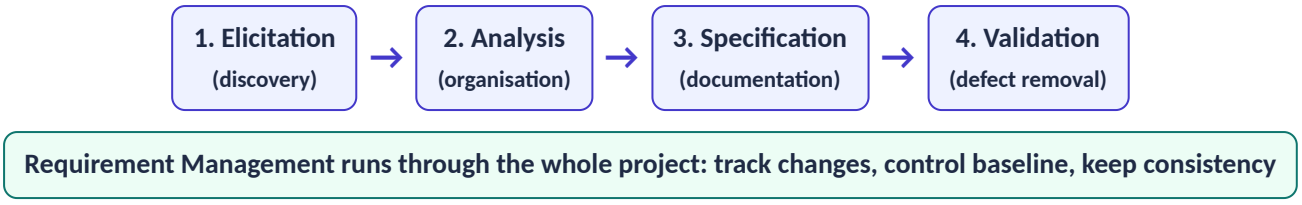


Figure: Requirements engineering process (Development + Management)

Requirements Elicitation

> **DEFINITION**
> Elicitation means **collecting requirements from different people** (stakeholders) to understand the real customer needs.

Goals: (1) find all parties who are sources of requirements, (2) collect requirements from them, (3) write them in original form, (4) combine them together.

Sources / Stakeholders: end users, clients and customers, managers, sponsors, developers, vendors and suppliers, domain experts, business policies, standards and guidelines. The **system analyst** talks to all of them.

Challenges in elicitation

- **Problem scope** is not clear (what is inside / outside the system).
- **Stakeholders** are many, and it is hard to identify all of them.
- **Understanding**: users cannot explain everything at once; the analyst may not know the domain.
- **Volatility**: requirements change again and again.
- Different stakeholders may have **conflicting** views.

Fact-finding techniques

Technique	How it works
Interview	Analyst asks questions face to face. Good for detail; takes time.
Questionnaire	Written questions sent to many people. Fast and cheap; less detail.
JAD (Joint Application Development)	Users, analysts and developers meet in a workshop and decide requirements together.
Onsite observation	Analyst watches how work is actually done in the organisation.
Prototyping	A sample model is shown to users to get feedback.
Viewpoints	Requirements are collected from the point of view of each type of user.
Review of records	Study old forms, reports, manuals and documents.

3.3 Requirements Analysis

In analysis we study the collected needs, remove confusion, and organise them. Main activities: **classification, organisation, prioritisation, negotiation and modelling**. Draft requirements are checked for correctness and completeness.

Analysis techniques: Structured analysis, Data-oriented analysis, Object-oriented analysis, Prototyping.

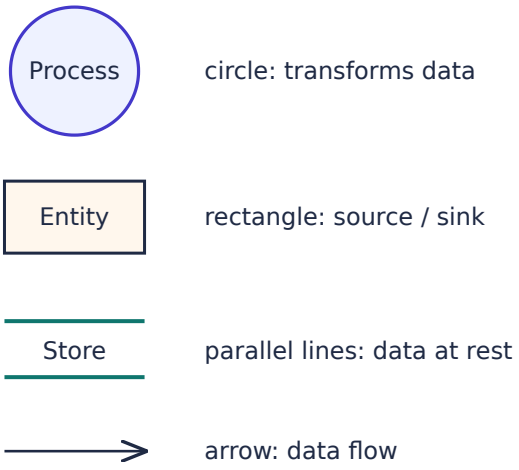
A. Structured Analysis (process modelling)

> **IDEA**
> Structured analysis looks at the system as **a process that transforms data**. It uses **top-down decomposition**: start with the whole system as one process, then break it into smaller sub-processes. Its tools are the **Data Flow Diagram (DFD)** and the **Data Dictionary**.

Data Flow Diagram (DFD)

A DFD is a picture that shows how **data moves** through a system and which functions work on it. It is also called a *bubble chart*, *process model* or *information flow model*. It does **not** show time or control (when a process runs).

Symbol	Meaning
Process (circle)	Transforms input data into output data. Named as verb + noun.
Data flow (arrow)	Movement of data between process, store and entity.
Data store (open box / parallel lines)	Data at rest, like a file or database.
External entity (rectangle)	Source or sink of data outside the system (student, customer).



DFD symbols as used in these notes

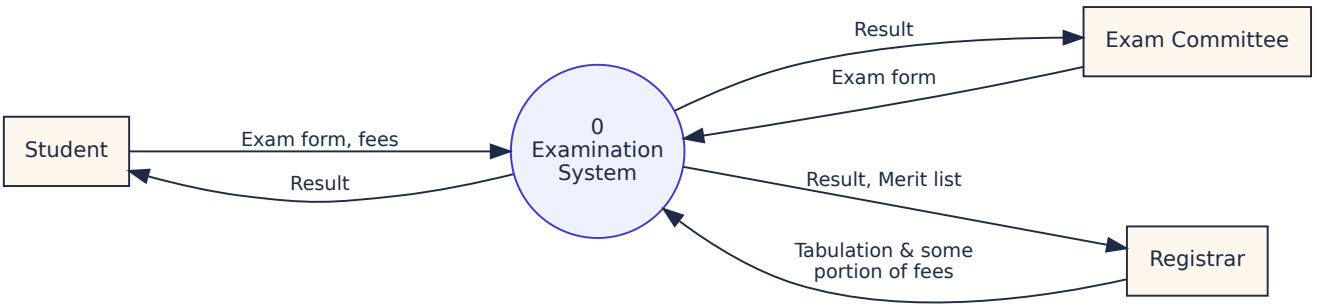
Levels of DFD (leveling)

- **Level 0 / Context diagram:** the whole system as *one bubble* with its external entities and main inputs/outputs.
- **Level 1:** main functions of the system (processes 1, 2, 3...).
- **Level 2:** each Level-1 process is broken into sub-processes (2.1, 2.2...). Level 2 is the *child* of Level 1. Continue until a process cannot be divided more.

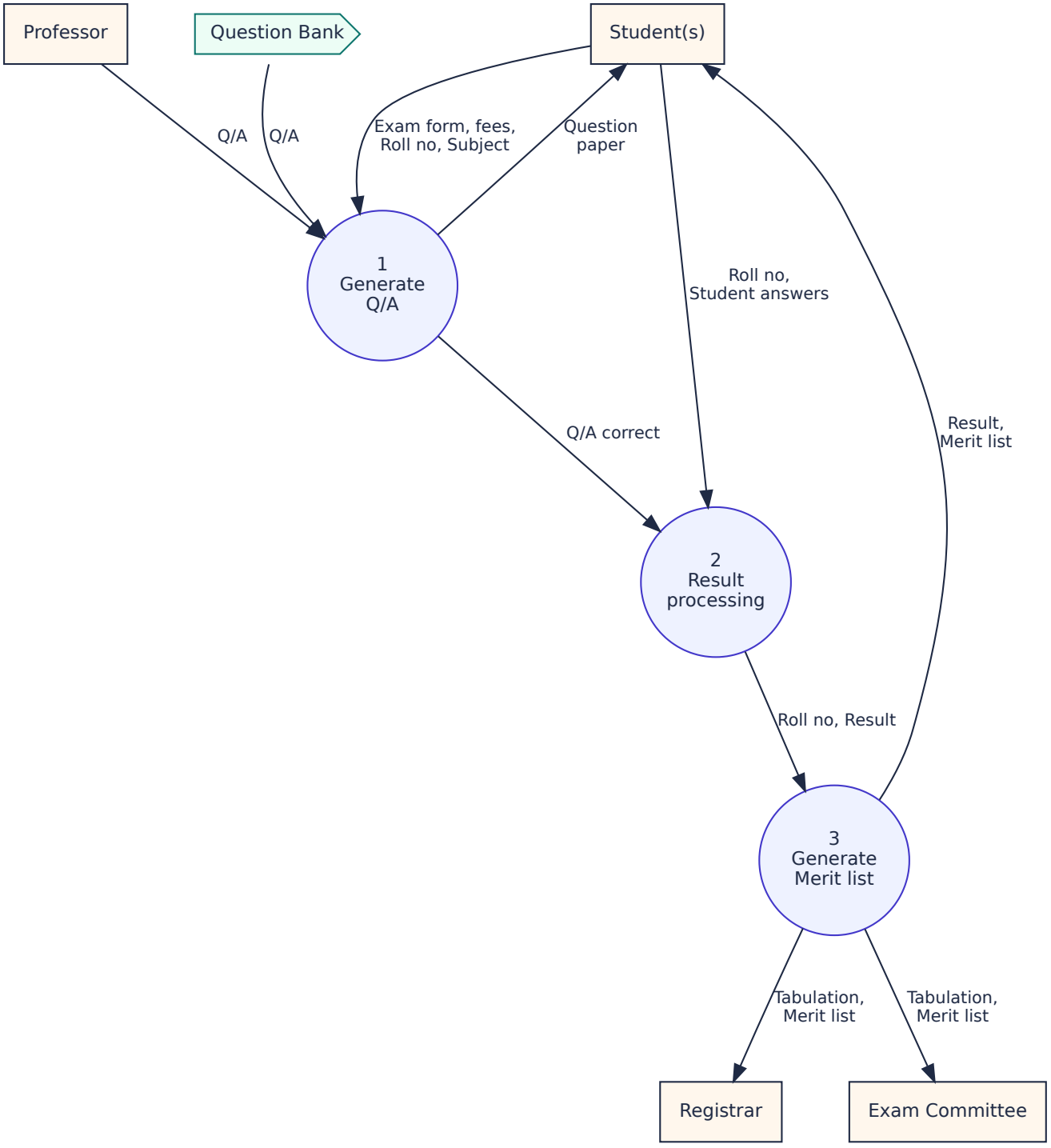
> **DFD RULES (WRITE THESE IN EXAM)**
> 1) Number each level (0, 1, 2). 2) Process names = verb + noun (e.g. *Generate_Merit_List*); entities and data flows = nouns. 3) Every process must have **at least one input and one output** (no black hole: only input; no miracle: only output). 4) Do not show control logic or loops. 5) **Balancing:** inputs and outputs of a parent process must match its child diagram. 6) Data cannot flow directly between two external entities or two data stores.

Mam's example: Examination System (DFD Level 0, 1, 2)

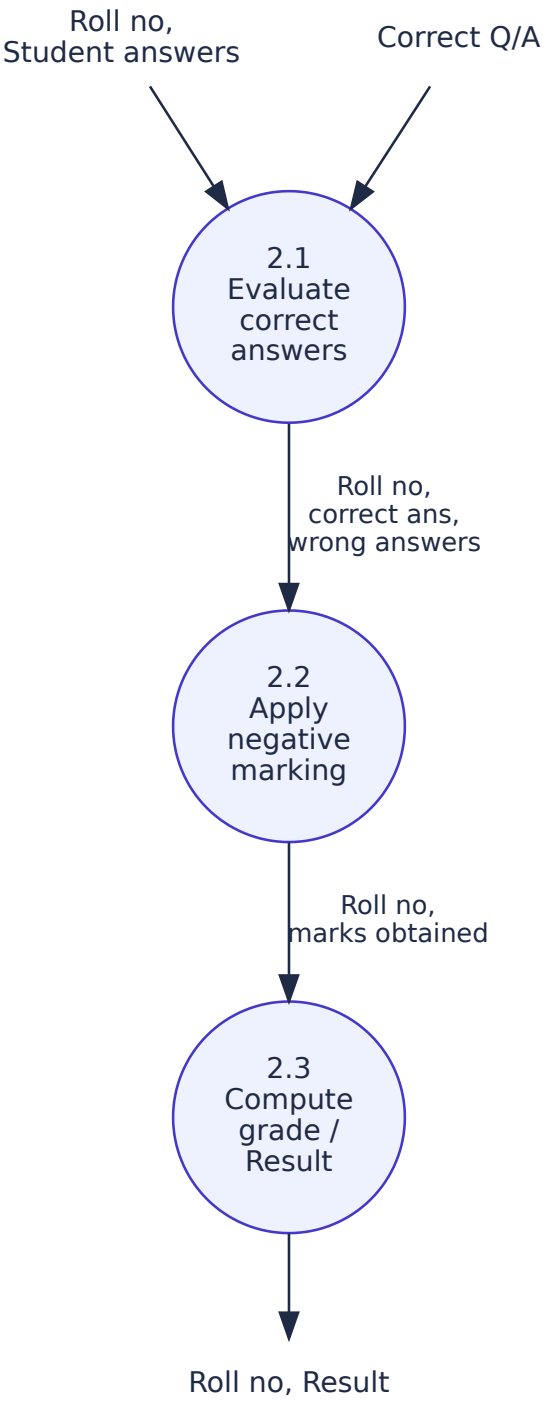
This is the diagram your mam shared. Learn the three levels, they are the most likely DFD question.



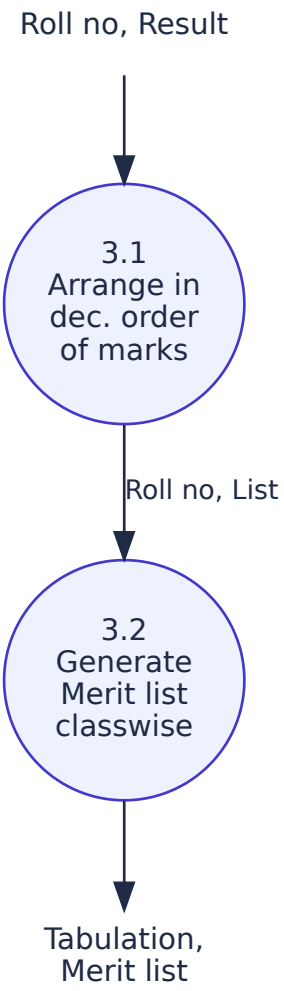
Level 0 (Context diagram) — Examination System as one process



Level 1 DFD — three processes: Generate Q/A, Result processing, Generate Merit list



Level 2 DFD for process 2 (Result processing)

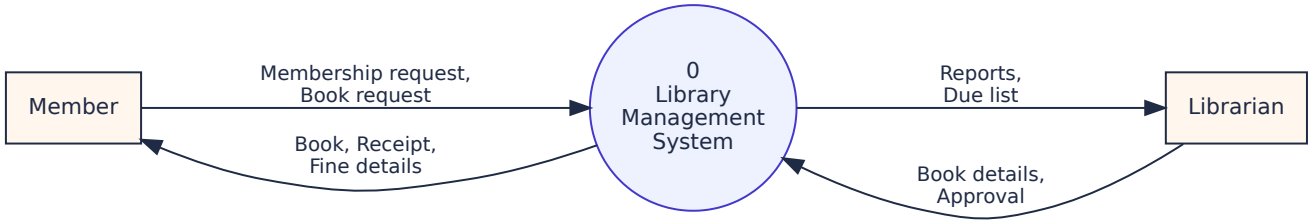


Level 2 DFD for process 3 (Generate Merit list)

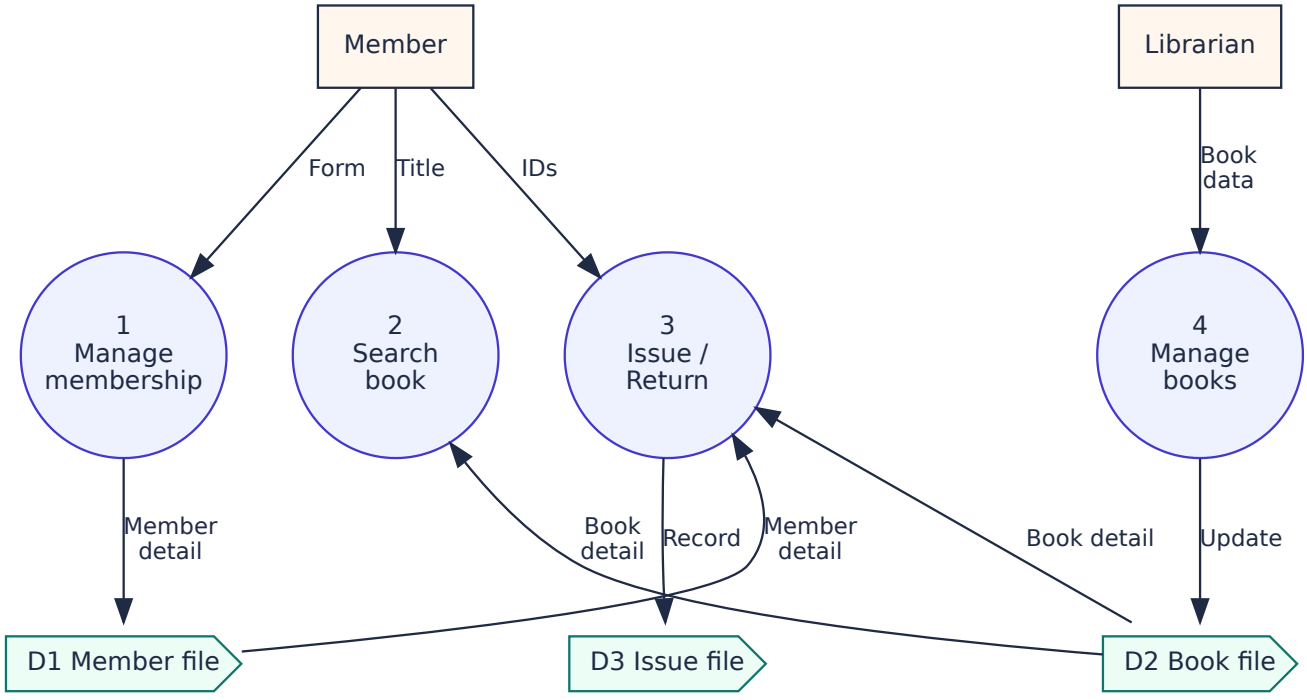
HOW TO EXPLAIN IT IN THE ANSWER

Context: Student sends exam form and fees and gets the result. Exam Committee sends exam form and gets result. Registrar gets result and merit list and gives tabulation and a part of the fees.
Level 1: (1) Generate Q/A uses questions from the Question Bank and Professor and gives the question paper to the student. (2) Result processing takes student answers and correct Q/A and produces the result. (3) Generate Merit list takes roll no and result and sends tabulation and merit list to Registrar and Exam Committee, and the result to the student.
Level 2 of process 2: 2.1 Evaluate correct answers → 2.2 Apply negative marking → 2.3 Compute grade/result. **Level 2 of process 3:** 3.1 Arrange in decreasing order of marks → 3.2 Generate merit list class-wise.

Another DFD example: Library Management System



Context diagram (Level 0) — Library Management System



Level 1 DFD — four processes and three data stores

Data Dictionary

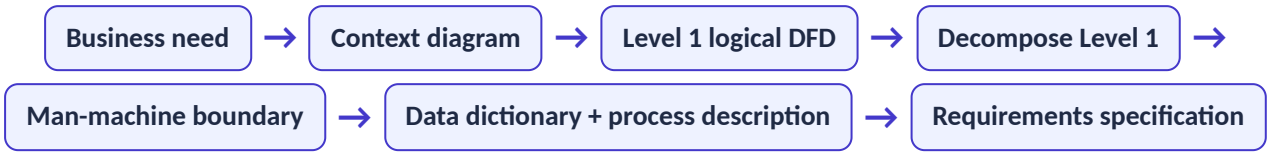
A data dictionary is **metadata**: it describes the data flows and data stores used in the DFD (long data names cannot be written on the arrows). It uses these symbols:

Symbol	Meaning
=	is made of / defined as
+	composition (and)
[a | b]	selection (either a or b)
{ ... }* or [...]*	repetition (many times)
( )	optional item

EXAMPLE (FROM SLIDES)

```
restaurant_bill = dish_price + sales_tax + service_charge + seat_charge +
grand_total
dish_price      = [dish_name + quantity + price]*
seat_charge     = [reserved_table | common_table]
```

Steps of the Structured Analysis method



Man-machine boundary: decide which parts will be done by the computer and which will stay manual (e.g. in an ATM the user selects options and takes cash; the machine does the rest).

ADVANTAGES

- Easy pictorial view for the customer
- Top-down, organised model
- Little technical knowledge needed
- Shows system scope and boundary
- Easy to convert into structured design

LIMITATIONS

- Final DFD is hard to read; does not show sequence
- Many DFD levels are time-consuming
- Needs requirements frozen early (waterfall style)
- Needs all requirements at the start

DFD vs Flowchart

DFD	Flowchart
Shows flow of **data**	Shows flow of **control** (steps of a program)
No branching or looping	Has decisions and loops
Can show parallel processes	Only one step active at a time
Shows data stores, inputs, outputs	Does not show data storage
Used in analysis (what)	Used in design / coding (how)

B. Object-Oriented Analysis (OOA)

OOA starts from the **problem statement** and builds an **object model**: it finds **objects/classes**, their **attributes**, **operations** and **relationships**. The model is drawn with class diagrams and use case diagrams (see Unit IV). Simple trick: **nouns in the problem statement → classes / attributes; verbs → operations**.

> **EXAMPLE**
> "A member issues a book from the library." Nouns: Member, Book, Library (classes). Verb: issue (operation *issueBook()*).

Structured vs Object-Oriented Analysis

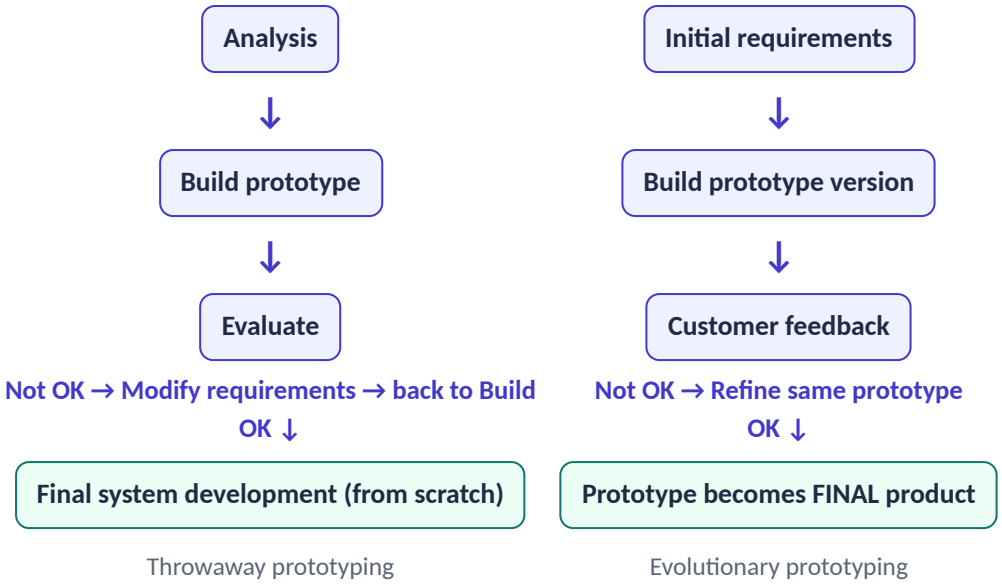
Point	Structured Analysis	Object-Oriented Analysis
Focus	Processes / functions	Objects (data + functions together)
Approach	Top-down decomposition	Bottom-up / identify objects and relations
Tools	DFD, data dictionary, ER diagram	Use case, class, object diagrams (UML)
Data and function	Kept separate	Combined in a class (encapsulation)
Reuse	Low	High (inheritance, polymorphism)
Change	A change may affect many functions	Changes stay inside the class
Best for	Data-processing, clear fixed requirements	Large, changing, complex systems

C. Prototyping Analysis

Prototyping is used when **requirements are not clear**, fast delivery is needed, and the customer must be involved. A **prototype** is a partly working model shown to the customer to get feedback. It is repeated until the customer is satisfied. It can be built using tools (Visual Basic, PHP, 4GL) or even **paper sketches**.

> **1. THROWAWAY PROTOTYPING**
> Built **quickly** only to check if the idea is right and to understand requirements. After feedback it is **discarded**; the real system is built from scratch.

> **2. EVOLUTIONARY PROTOTYPING**
> The prototype is built carefully and **keeps improving** after each feedback until it becomes the **final system**. Uses RAD and CASE tools.



Throwaway	Evolutionary
Prototype is thrown away	Prototype becomes the final system
Quick and rough, little quality	Built with good quality
Best when requirements are very unclear	Best when requirements are partly known
Documentation can be skipped	Needs proper documentation

3.4 Requirements Specification (SRS)

> **SRS DEFINITION**
> The SRS is a formal document that gives a **complete description of the proposed software: WHAT it will do, not HOW**.

Why do we need an SRS?

- Customers understand a written document better than technical models.
- It is the **base** for design, coding, testing, delivery and maintenance.
- It is the **agreement** between customer and supplier.
- It is the reference to verify and validate the final software.
- A good SRS reduces cost, time and effort, because unclear requirements cause failed projects.

Characteristics of a good SRS

Correct, Unambiguous, Complete, Consistent, Ranked for importance/stability, Verifiable, Modifiable, Testable, Valid, Traceable.

Components of an SRS

Functional requirements, performance requirements, design constraints (hardware/software), and external interface requirements.

IEEE structure of SRS (short form)

1. Introduction	2. General description	3. Specific requirements
1.1 Purpose 1.2 Scope 1.3 Definitions, acronyms 1.4 References 1.5 Document overview	2.1 Product perspective 2.2 Product functions 2.3 User characteristics 2.4 General constraints 2.5 Assumptions and dependencies	3.1 Functional requirements (input, processing, output) 3.2 External interfaces 3.3 Performance 3.4 Design constraints 3.5 Security, reliability, availability, maintainability, database, safety, operational

Requirements specification methods

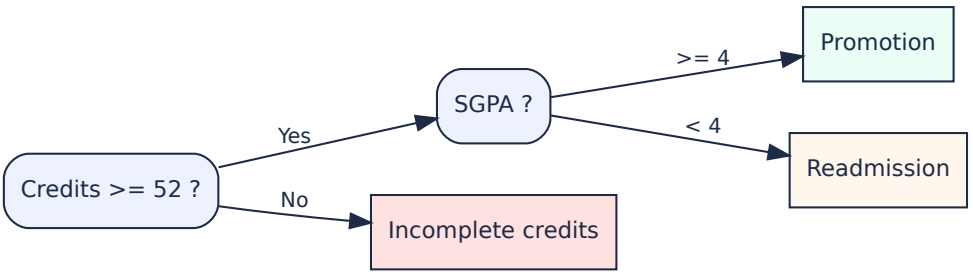
Natural language is **ambiguous** (different people understand it differently), so complex requirements are written using these methods:

1. Decision tree and decision table

> **EXAMPLE: STUDENT PROMOTION SYSTEM (FROM SLIDES)**
> A student must complete **52 credits** and have **SGPA ≥ 4.00** to go to the next session. Otherwise he is re-admitted in the same course.

Condition / Action	R1	R2	R3
Credits ≥ 52	No	Yes	Yes
SGPA ≥ 4	–	No	Yes
Incomplete credits	×		
Readmission		×	
Promotion			×

Decision table: top part = conditions (stub), bottom part = actions. R = rule.



Decision tree: edges = conditions, leaves = actions

2. Program Design Language (PDL)

Requirements written as **pseudo-code** in structured English. Easy to turn into a program, but hard to express domain ideas and it looks more like design than specification.

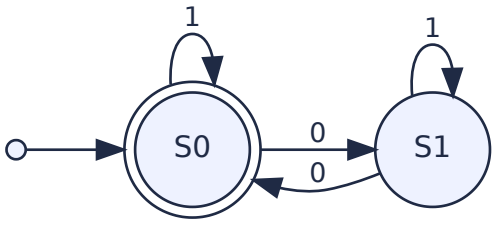
```
Procedure SPS is BEGIN
  LOOP
    read student_result(credits, SGPA)
    IF credits >= 52 THEN calculate SGPA ELSE message "incomplete credits" ENDIF
    IF SGPA >= 4.00 THEN promote() ELSE readmit() ENDIF
  END LOOP
END
```

3. Regular expressions

A pattern that describes a set of strings. Operators: `r1 | r2` (or), `r1 r2` (join), `r*` (repeat). Example: `a*ba*` means {b, aba, aabaa ...}. Used for searching, lexical analysis, input formats.

4. Graphical methods — DFD, ER model and Finite State Machine (FSM)

An FSM has **states, transitions, conditions/rules and input events**, with an initial state and final state(s). Good for real-time software, but becomes huge for big systems.



FSM that accepts an even number of 0s (S0 is the start and final state)

State	Input 0	Input 1
S0 (start, final)	S1	S0
S1	S0	S1

5. Mathematical (formal) methods

Specification is written in mathematics, so it is **clear, unambiguous and provable**, but difficult for customers to understand. Algebraic specification = **(Sorts, Operations, Axioms)**.

> **EXAMPLE: STACK**
> Sort: stack, element. Operations: `push: stack x element → stack`, `pop: stack → stack`, `top: stack → element`. Axiom: `top(push(s, e)) = e`.

3.5 Requirements Validation and Management

> **VALIDATION**
> Validation checks that the SRS **correctly and clearly** represents the customer needs. It finds **omissions, inconsistency, wrong facts and ambiguity**. Methods: **requirements review, inspection, test-case generation, reading, prototyping**.

> **MANAGEMENT**
> Requirements management means **collecting, organising, documenting, prioritising and negotiating** requirements during the whole project, because they keep changing. Main activities: plan requirements; identify them; **manage changes**; control and track changes; get agreement among stakeholders; regular reviews; **impact analysis** of a change.

Requirements management tools

Tools help to store requirements in one place, give each requirement a unique ID, link it to design/code/tests (**traceability**), track versions and approvals. Common tools: **IBM DOORS, Jama Connect, Jira, Azure DevOps, Confluence** and spreadsheets for small projects.

Software Design

Unit III (continued)

3.6 Software Design

DEFINITION

Requirements say **WHAT** to build. **Design says HOW** to build it. Design converts the SRS into a blueprint (structure, modules, data, interfaces) that programmers can code.

Design fundamentals (key concepts)

Concept	Meaning (simple)
Abstraction	Show only the important details, hide the rest. Example: you press "withdraw"; you need not know how the bank database works.
Modularity	Divide the software into small independent parts (modules). Easy to build, test and change.
Information hiding	A module hides its internal data and shows only an interface.
Stepwise refinement	Start with a general statement and refine it step by step into detail (top-down).
Architecture	Overall structure of the system: main components and how they connect.
Coupling and cohesion	Measures of module quality. Aim: **low coupling, high cohesion**.
Separation of concerns	Each part handles only one job (UI, logic, data are separate).

Design process



1. **Architectural design:** decide the main parts and how they communicate.
2. **Data design:** choose data structures and database design.
3. **Interface design:** user interface and communication between modules.
4. **Component-level (detailed) design:** write the logic of each component (pseudo-code, class details).
5. **Review** the design against the SRS.

Modular design: Coupling and Cohesion

COUPLING

How strongly **one module depends on another**. Lower is better.

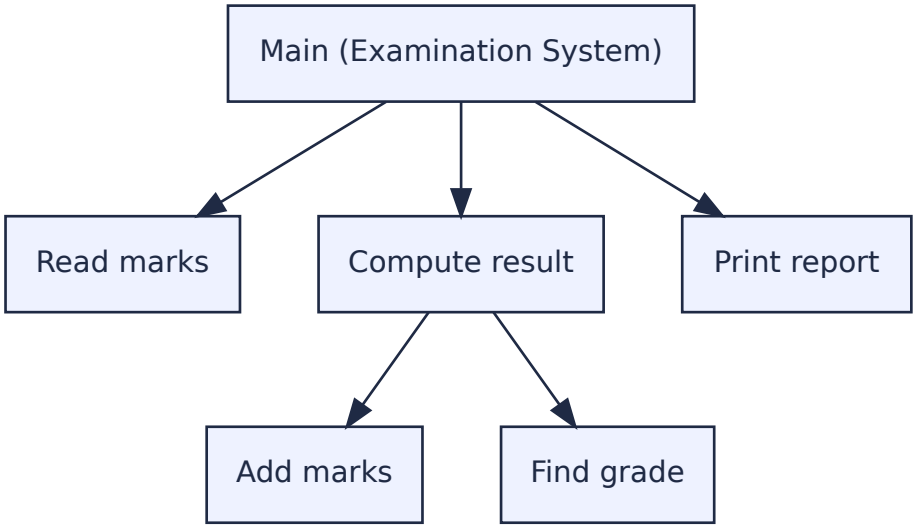
Best → worst: Data → Stamp → Control → Common → Content.

COHESION

How strongly the **parts inside one module belong together**. Higher is better.

Worst → best: Coincidental → Logical → Temporal → Procedural → Communicational → Sequential → Functional.

Type	Easy meaning
Data coupling	Modules share only simple data through parameters (good).
Control coupling	One module passes a flag that controls the other's logic.
Common coupling	Modules share a global variable.
Content coupling	One module changes the inside of another (worst).
Coincidental cohesion	Unrelated tasks thrown into one module (worst).
Functional cohesion	Every part works for one single task, e.g. *calculateFine()* (best).



Modular design as a hierarchy (structure chart): Main calls Read marks, Compute result and Print report; Compute result is further split. Each module has one job.

Component-level design

A **component** is an independent, replaceable part of the system that gives services through interfaces (for example the Book Management component). Component-level design describes **the inside of every component**: its classes, attributes, operations, interfaces, and the algorithm of each operation.

GOOD DESIGN GUIDELINES

- **Open-Closed Principle:** open for extension, closed for change.
- **Single responsibility:** a component should do one job.
- **Liskov substitution:** a subclass should work wherever its parent works.
- **Dependency inversion:** depend on interfaces, not on concrete classes.
- Keep coupling low and cohesion high; use clear interfaces.

EXAMPLE

Component *Member Management* has classes Member and MembershipForm, and offers the interface *addMember()*, *deleteMember()*. Other components (Library Main System) only call this interface; they never touch its internal data.

3.7 Structured vs Object-Oriented Design, and Refactoring

Point	Structured design	Object-oriented design
Basic unit	Function / procedure (module)	Object / class
Starts from	DFD and data dictionary	Use cases and class model
Output	Structure chart, module specification	Class, sequence, state diagrams
Data	Global data shared by functions	Data hidden inside objects
Decomposition	By function (top-down)	By objects
Reuse and change	Harder	Easier (inheritance, polymorphism)
Quality goal	Low coupling, high cohesion of modules	Encapsulation, loose coupling of classes

Refactoring

DEFINITION

Refactoring means **improving the internal structure of existing code or design without changing its external behaviour**. Output stays the same; code becomes cleaner.

Why? Remove duplicate code, make code easier to read and change, reduce bugs and technical debt.
When? When you see "code smells": duplicate code, very long methods, very large classes, too many parameters, unclear names.

Technique	What it does
Extract method	Move a block of code into its own named method.
Rename method / variable	Give a meaningful name.
Move method / field	Move it to the class where it belongs.
Extract class	Split a big class into two.
Replace conditional with polymorphism	Remove long if/else using subclasses.

EXAMPLE — EXTRACT METHOD

```
Before:  printBill() { ...print header...  total = price*qty + tax;  ...print
total... }
After:   printBill() { printHeader();  total = calculateTotal();
printTotal(total); }
```

The output is the same, but each part is now small and reusable. Always **re-run tests** after refactoring.

3.8 Probable questions — Unit III

Q1 (5 MARKS)
Draw the context diagram, Level 1 and Level 2 DFD for an **Examination System**. → See section 3.3 (mam's example).

Q2 (5 MARKS)
Explain the requirements engineering process with a diagram. → 3.2 (five phases: elicitation, analysis, specification, validation, management).

Q3 (5 MARKS)
Differentiate functional and non-functional requirements with examples. → 3.1.

Q4 (5 MARKS)
What is DFD? Explain its symbols, levels and rules. Differentiate DFD from a flowchart. → 3.3.

Q5 (5 MARKS)
Compare structured analysis and object-oriented analysis. → 3.3 B.

Q6 (5 MARKS)
Explain throwaway and evolutionary prototyping. → 3.3 C.

Q7 (5 MARKS)
What is an SRS? Explain its characteristics and IEEE structure. → 3.4.

Q8 (5 MARKS)
Represent a given rule using decision tree and decision table (student promotion, ATM withdraw, discount rule). → 3.4.

Q9 (5 MARKS)
Explain coupling and cohesion with types. → 3.6.

Q10 (5 MARKS)
What is refactoring? Give an example. Differentiate structured and OO design. → 3.7.

UNIT IV

Object-Oriented Analysis and Design • UML

4.1 Object-oriented basics

Object

> **DEFINITION**
> An **object** is a thing that exists in the real world or in an abstract way: a student, a train, an invoice, a file. Every object has **state, behaviour and identity**.

Part	Meaning	Example: object "Joe"
State	Attributes and their current values. Values can change with time.	name = Joe, age = 25, qualification = MCA
Behaviour	How an object acts and reacts (operations and message passing). State and operations together define behaviour.	Joe can study, teach, pay fees depending on his role
Identity	Property that makes an object **unique**, different from all others even if values are same.	Roll no / ID 101

Class

> **DEFINITION**
> A **class** is a **blueprint / template** for a set of objects that share the same attributes, operations, relationships and meaning. Object = instance of a class. Example: class *Student*; objects *rahul*, *priya*.

- **Attribute:** data held by an object. **Operation:** a service the class provides (implemented as a method). A **signature** = operation name + number/type of arguments + return type.
- **Visibility:** + public (all can use), - private (only the class itself), # protected (class and its subclasses).

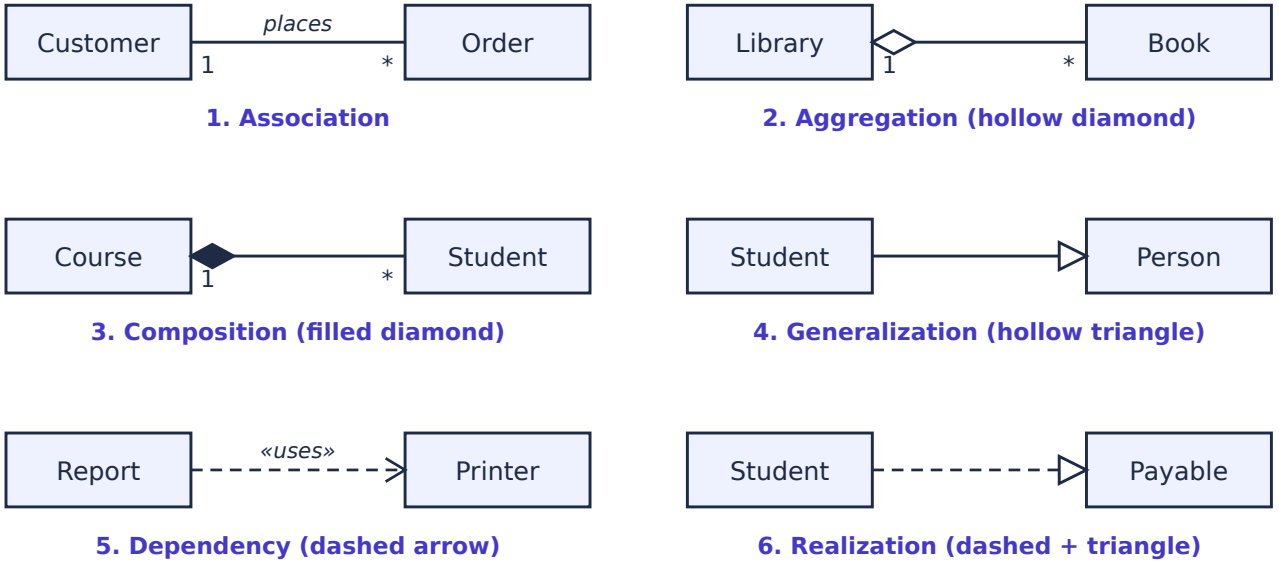
Four main OO features

Feature	Meaning	Example
Encapsulation and **data hiding**	Binding attributes and methods together in a class. Data is private and is accessed only through public methods.	Account balance is private; use *getBalance()*, *deposit()*.
Inheritance	A subclass gets attributes and operations of a superclass. Gives reuse and "is-a" relation.	Student and Teacher inherit from Person.
Polymorphism	"Many forms": the same operation behaves differently on different objects/data (overriding, overloading).	*add()* adds numbers, joins strings, merges files.

Message passing	Objects communicate by sending messages (request to run an operation). Implemented as method calls.	*store.getItem(itemID)*

Types of inheritance: Single (one parent), Multiple (many parents), Multilevel (chain: A→B→C), Hierarchical (one parent, many children), Hybrid (mix of multiple + multilevel).

Relationships between classes / objects



Association, Aggregation (hollow diamond), Composition (filled diamond), Generalization (hollow triangle), Dependency (dashed arrow)

Relationship	Meaning	Symbol	Example
Association	A link between classes ("knows / uses"). Can be binary, ternary, n-ary. Has **multiplicity** (1, 0..1, *, 1..*), role names, and can have attributes.	Plain line	Customer places Order
Aggregation	"Whole-part", but the part can **live without** the whole ("has-a", weak).	Hollow diamond at the whole	Library has Books; if the library closes, books still exist
Composition	Strong whole-part. Part **dies with the whole**; the whole owns the creation and destruction of parts.	Filled diamond at the whole	Course has Registrations; Building has Rooms
Generalization	"Is-a" relation between parent and child (inheritance).	Solid line with hollow triangle at the parent	Student is a Person

Dependency	One class **uses** another; a change in one may affect the other. Weak, short-lived link. Stereotypes like «uses», «create».	Dashed arrow	Report uses Printer
Realization	One side gives a contract (interface), the other side **implements** it.	Dashed line with hollow triangle	Student realizes the Fees interface

AGGREGATION VS COMPOSITION (VERY COMMON QUESTION)

Aggregation: part can exist alone (hollow diamond). Composition: part cannot exist without the whole, lifetime is same (filled diamond). Think: **Library–Book = aggregation, House–Room = composition.**

4.2 Unified Modeling Language (UML)

DEFINITION

UML is a **standard visual language for specifying, visualising, constructing and documenting** software systems. It is a **modelling language, not a programming language**. First published by the **Object Management Group (OMG) in 1997**.

History / evolution

UML was created by combining three methods: **Booch method** (Grady Booch: good for design and implementation), **OMT** (James Rumbaugh: analysis and data-intensive systems) and **OOSE / Objectory** (Ivar Jacobson: use cases and requirements). These three people are called the "three amigos" and worked at Rational Software.

Year	Version
1995	Unified Method 0.8
1996	UML 0.9 and 0.91
1997	UML 1.0, 1.1 (adopted by OMG)
1999 / 2001	UML 1.3 / 1.4
2003	UML 2.0 (current versions are 2.x)

Tools: Rational Rose (IBM), StarUML, ArgoUML, Umbrello, Visual Paradigm, Microsoft Visio.

Conceptual model of UML

Three major elements: **Building blocks, Rules, Common mechanisms**.

1. Building blocks: Things, Relationships, Diagrams

Things	Meaning	Examples
Structural	"Nouns": static parts of the model	Class, interface, collaboration, use case, active class, component, node
Behavioural	"Verbs": dynamic behaviour	Interaction, state machine
Grouping	Organisational parts	Package
Annotational	Explanations	Note (rectangle with folded corner)

Relationships (4): Dependency, Association, Generalization, Realization.

Diagrams (9): Class, Object, Use case, Sequence, Collaboration, State chart, Activity, Component, Deployment.

2. Rules

Rules keep the model consistent. Semantic rules for: **Names, Scope, Visibility, Integrity, Execution**.

3. Common mechanisms

- **Specifications:** a text description behind every graphical symbol.
- **Adornments:** extra marks on a symbol (name, visibility, multiplicity).
- **Common divisions:** class vs object; interface vs implementation.
- **Extensibility: Stereotypes** (new kind of element, written « »), **Tagged values**, **Constraints** (rules in { }).

UML notations of important elements

Element	Notation and purpose
Actor	Stick figure. A user or external system that interacts with the system.
Use case	Ellipse with the use case name. A function the system gives to an actor.
Class	Rectangle with 3 compartments: name / attributes / operations.
Interface	Class with «interface» or a small circle (lollipop). Has only operations, no attributes.
Component	Rectangle with two small rectangles on its side. A replaceable software unit.
Package	Folder-like rectangle with a tab. Groups classes and other elements.
Node	3-D cube. A physical computing device (server, PC) used in deployment diagrams.
State	Rounded rectangle. A condition of an object.
Note	Rectangle with folded top-right corner. Comments and constraints.

4+1 view of UML

UML looks at a system from different views. The "+1" is the **user's view**, which cuts across all other views.

View	Also called	What it shows	Diagrams
User view (+1)	Analysis view	System functions, services and actors	Use case
Structural view	Design view	Classes, interfaces, relationships	Class, Object
Process view	Behavioural view	Flow and dynamic behaviour	Sequence, Collaboration, State chart, Activity
Implementation view	Physical view	Components assembled into the physical system	Component
Deployment view	Environmental view	Physical nodes where components run	Deployment

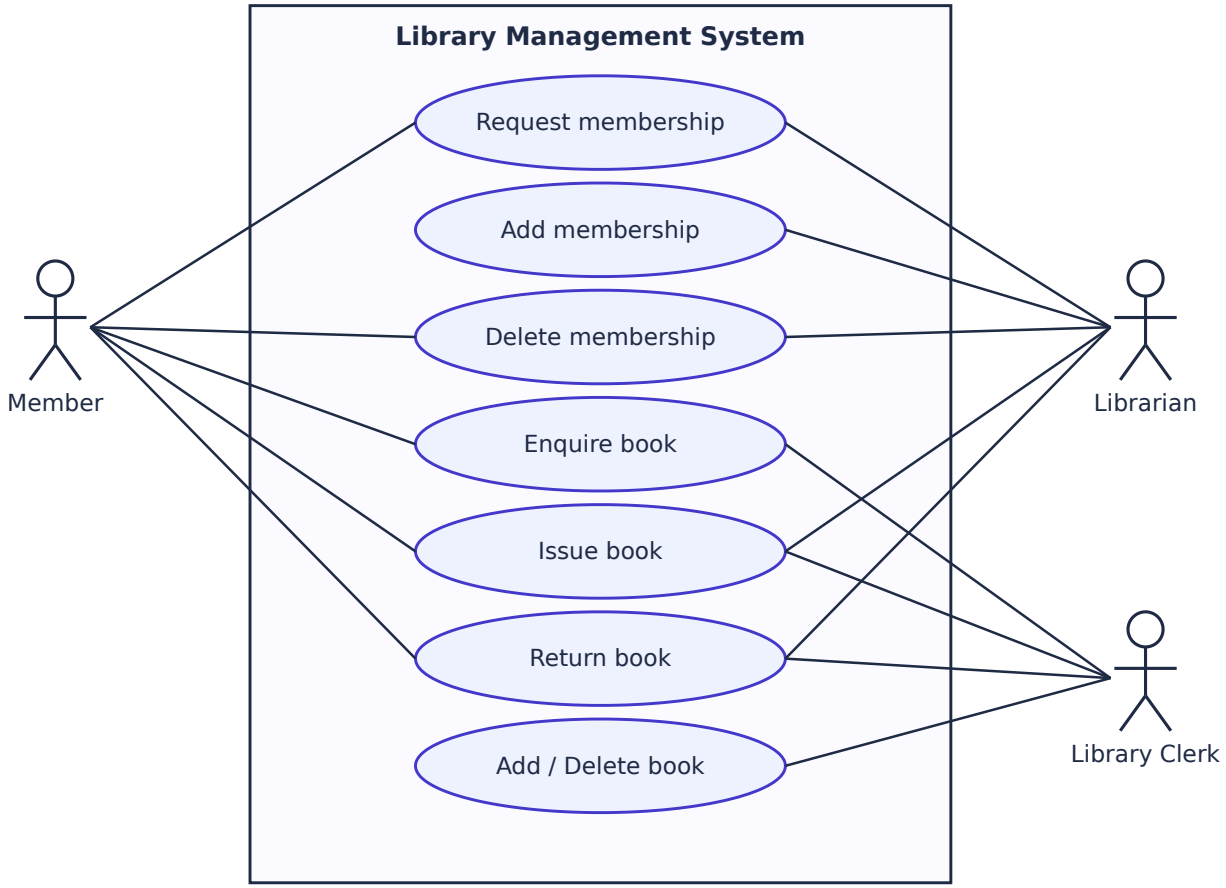
4.3 Use Case Diagram

> **DEFINITION**
> A **use case diagram** shows **what the system does for its users**. It shows **actors**, **use cases** and the lines between them. It is mainly used to **capture user requirements** at an abstract level (without implementation detail).

- **Actor:** a user or external system/event that interacts with the system, shown as a **stick figure**. Actors represent roles (Member, Librarian), not particular persons.
- **Use case:** a set of functions the system performs for one goal, shown as an **ellipse** with a verb-phrase name.
- **System boundary:** a rectangle around the use cases.
- **Communication line:** solid line joining actor and use case.

Steps to draw a use case diagram

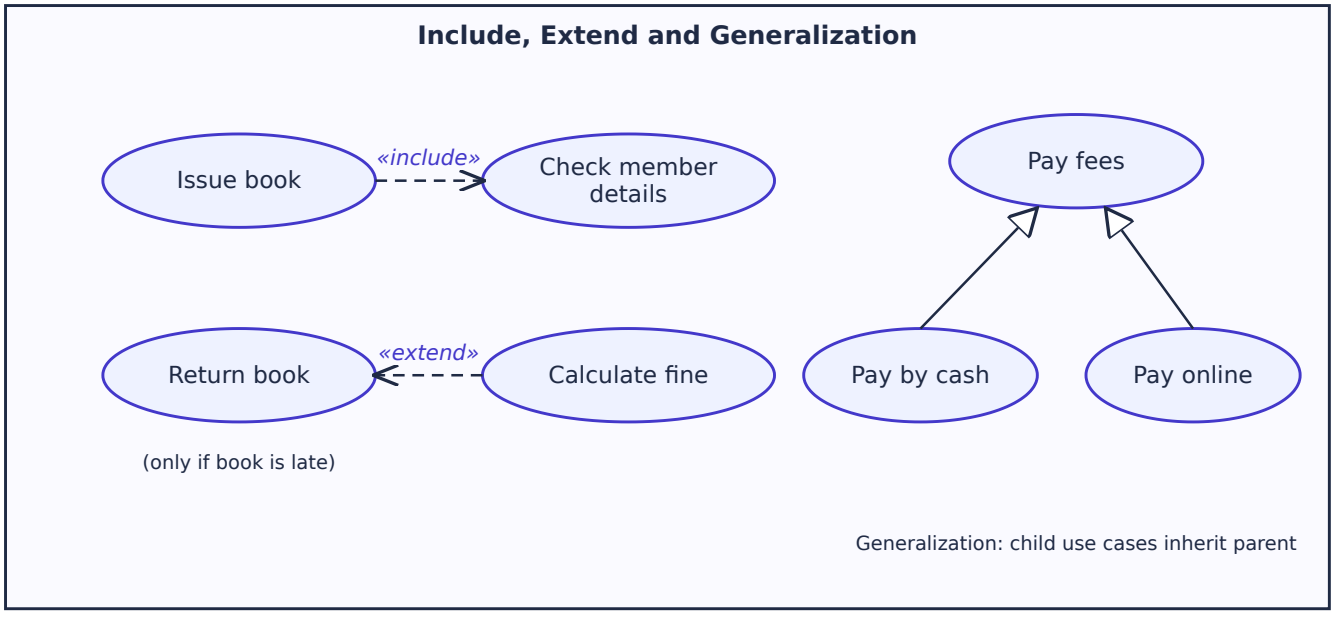
1. Identify the **system** and draw the boundary box.
2. Identify **actors** (who uses the system?).
3. Identify **use cases** (what does each actor want to do?).
4. Connect actors to use cases with lines.
5. Add **include / extend / generalization** where needed.



Use case diagram for Library Management System (the 8 scenarios of your slides: request, add, delete membership; enquire, issue, return; add / delete book)

Use case relationships

Relationship	Meaning	Notation
Communicates	Exchange of information between an actor and a use case.	Solid line
Include	Common behaviour is **always** used by another use case. Base use case needs it to be complete. Reduces repetition. (Issue book *includes* Check member)	Dashed arrow «include» from base to included
Extend	**Optional** or special behaviour added only under a condition. (Return book is *extended by* Calculate fine, only if late)	Dashed arrow «extend» from extension to base
Generalize	A child use case or actor inherits and specialises the parent. (Pay by cash / online are kinds of Pay fees)	Solid line with hollow triangle at parent



Include, extend and generalization in one picture

INCLUDE VS EXTEND

Include = **must happen** every time (arrow points to the included use case). Extend = **may happen** only on a condition (arrow points to the base use case).

Use case scenario (textual description)

A use case scenario describes a use case in words. It has these parts: **System, Use case name and number, Participating actors, Description, Preconditions** (true before it starts), **Post-conditions** (true after it ends), **Flow of events** (steps; sub-flows), **Non-functional requirements, Reference documents**.

Scenario: Issue book (from slides)	
System	Library management system
Use case name / number	Issue books / UC-05
Participating actors	Member and Librarian (clerk)
Description	Member requests to issue a book from the library clerk

Preconditions	Member is a registered member of the library
Post-conditions	Clerk issues the available book
Flow of events	1. Member requests the book. 2. Clerk provides the book if available in the database. 3. Clerk updates the system against the member ID.
Non-functional requirement	Only a limited number of books can be issued to a member
Reference documents	Issue process and guidelines

OTHER SCENARIOS IN YOUR SLIDES (JUST REMEMBER THE IDEA)

UC-01 Request membership: member asks, librarian gives form, member fills and submits. **UC-02 Add membership:** librarian submits form, system generates a unique membership ID. **UC-03 Delete membership:** allowed only if no dues. **UC-04 Enquire book:** clerk checks availability. **UC-06 Return book:** clerk checks return date and charges overdue fine. **UC-07 / 08 Add / Delete book:** clerk updates quantity; delete needs librarian's permission.

ETRANSQ — TENDER MANAGEMENT (UC-001)

Actors: Supplier, Agent, Transporter. **Description:** supplier posts tenders; transporters and agents search tenders with one click; the date of a published tender can be changed. **Precondition:** the user posting the tender is a supplier. **Post-conditions:** notification generated for all associates; supplier gets an email. **Flow:** 1. Supplier requests to upload a tender. 2. After filling the form, a PDF file is uploaded. 3. Tender is published on the given date. **Sub-flow:** search tender, publish tender, activate/deactivate tender.

4.4 Class Diagram and Object Diagram

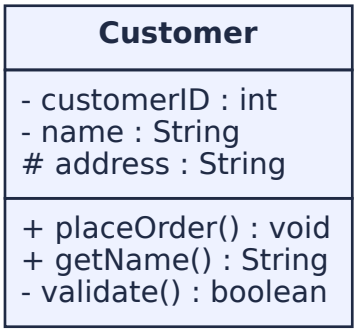
CLASS DIAGRAM

It shows the **static (structural) design** of the system: classes, their attributes and operations, and the **relationships** between them (association, aggregation, composition, generalization, dependency). It can be converted directly into code.

Class notation

- Top: **class name**
- Middle: **attributes** `visibility name : type`
- Bottom: **operations** `visibility name(params) : return`
- Visibility: + public, - private, # protected

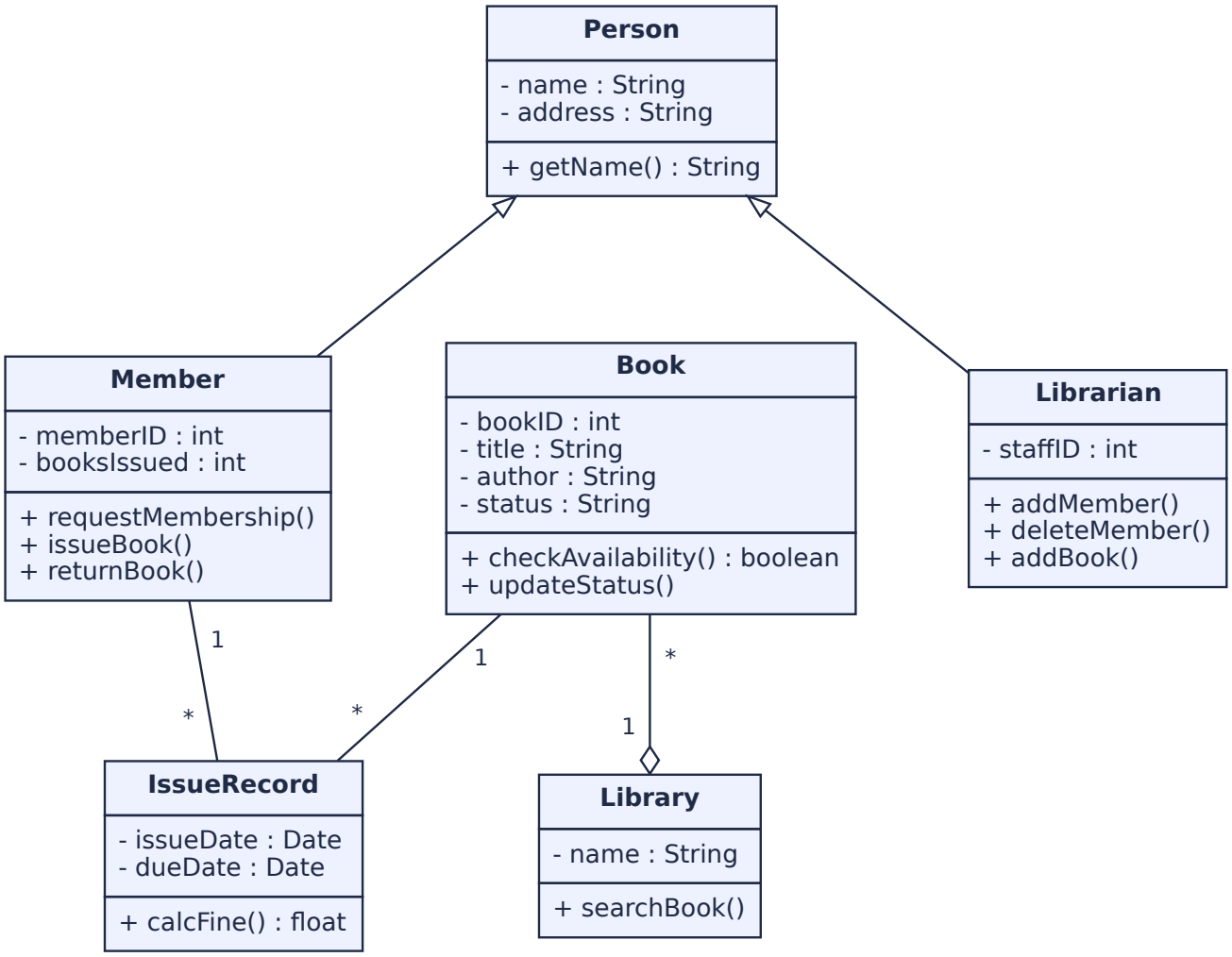
Multiplicity: 1 exactly one, `0..1` zero or one, * many, `1..*` one or more.



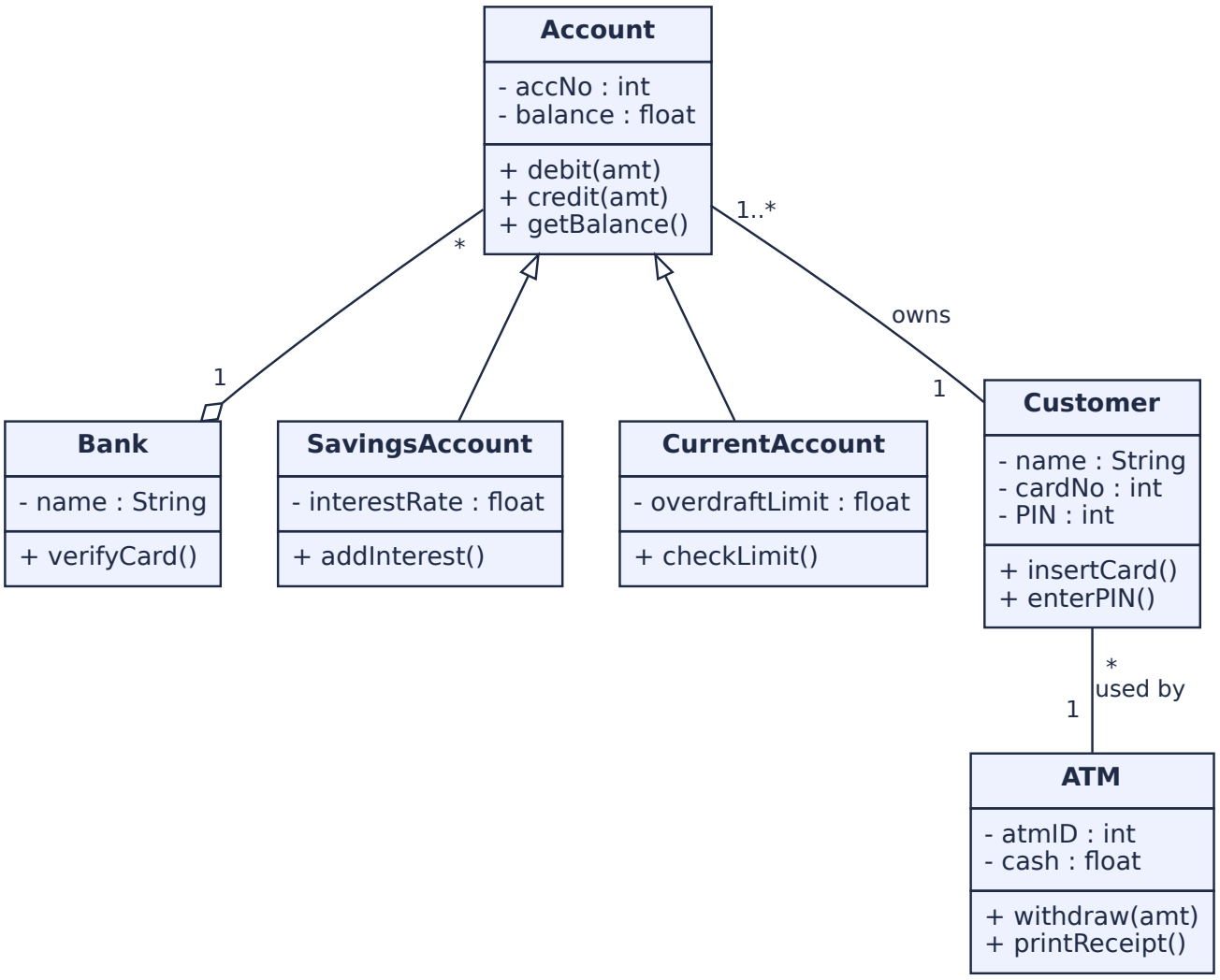
A class with three compartments

Steps to draw a class diagram

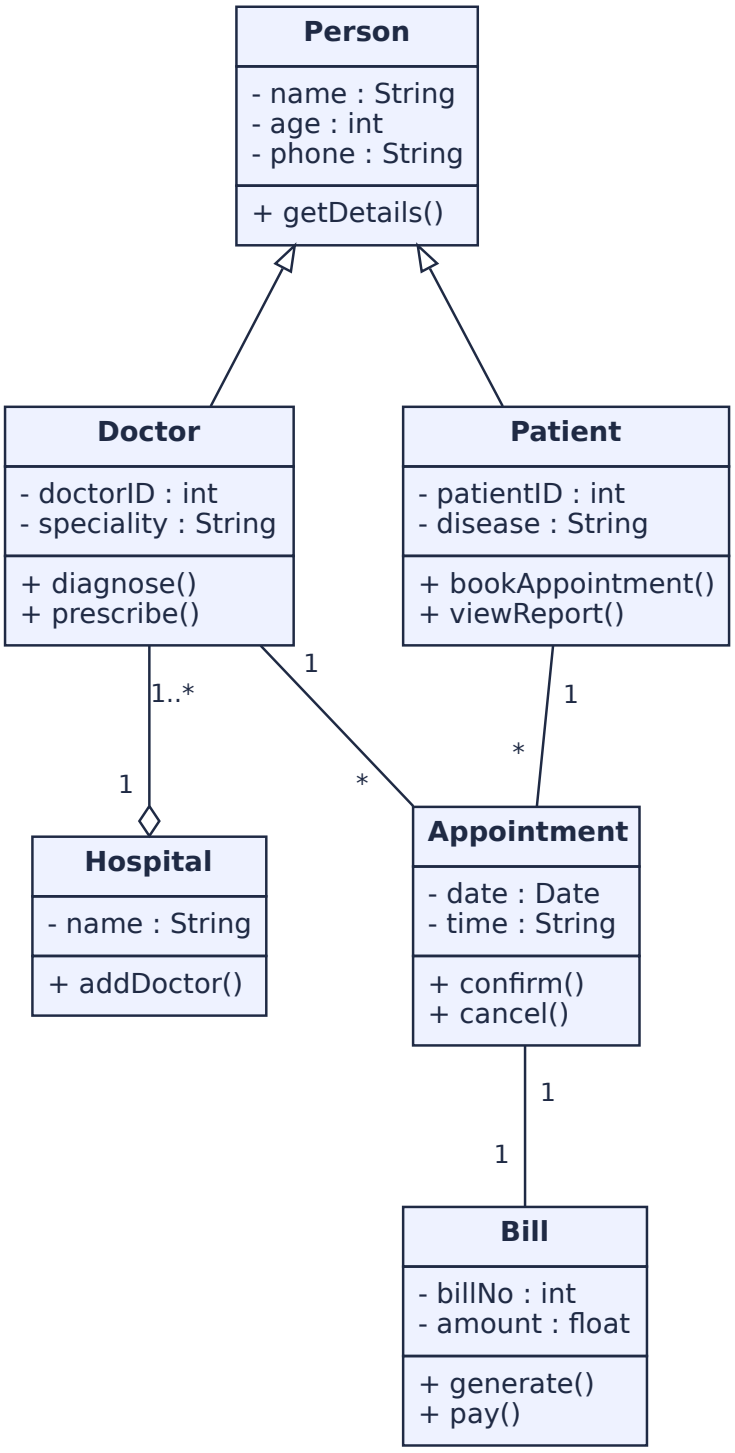
1. Find **classes** (nouns in the problem). 2. Add **attributes and operations**. 3. Find **relationships** and give multiplicity. 4. Use generalization for common parts. 5. Check that every class is needed.



Class diagram: Library Management System — Member and Librarian inherit from Person; IssueRecord links Member and Book; Library aggregates Books



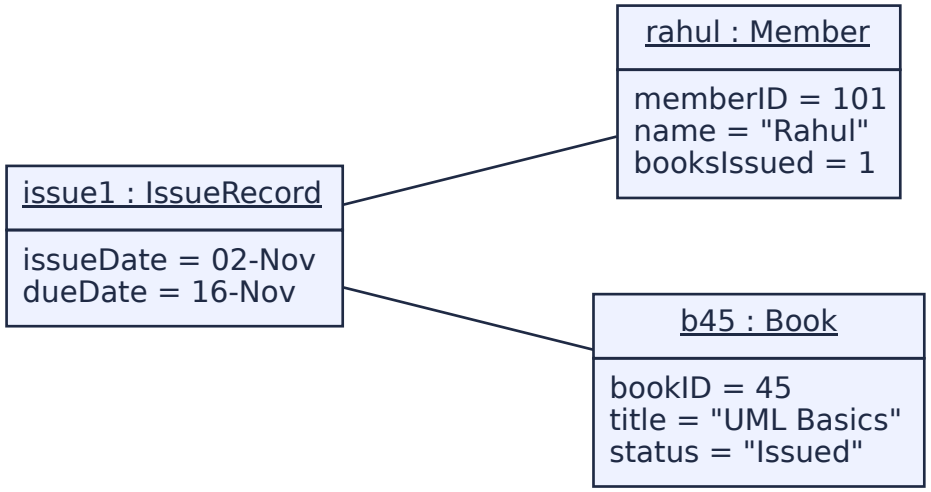
Class diagram: ATM system — two kinds of Account (generalization), Bank aggregates Accounts, Customer owns accounts and uses the ATM



Class diagram: Hospital Management — Patient and Doctor are Persons; Appointment connects them; Bill belongs to an Appointment

OBJECT DIAGRAM

An object diagram is an **instance of a class diagram**: a **snapshot of the system at one moment**, with real data values. Object name is underlined as `objectName : ClassName`. It is closer to the real running system, and it is used to check whether the class diagram is correct.



Object diagram: Library — Rahul (a Member) has issued the book "UML Basics" (an instance of Book) through issue1

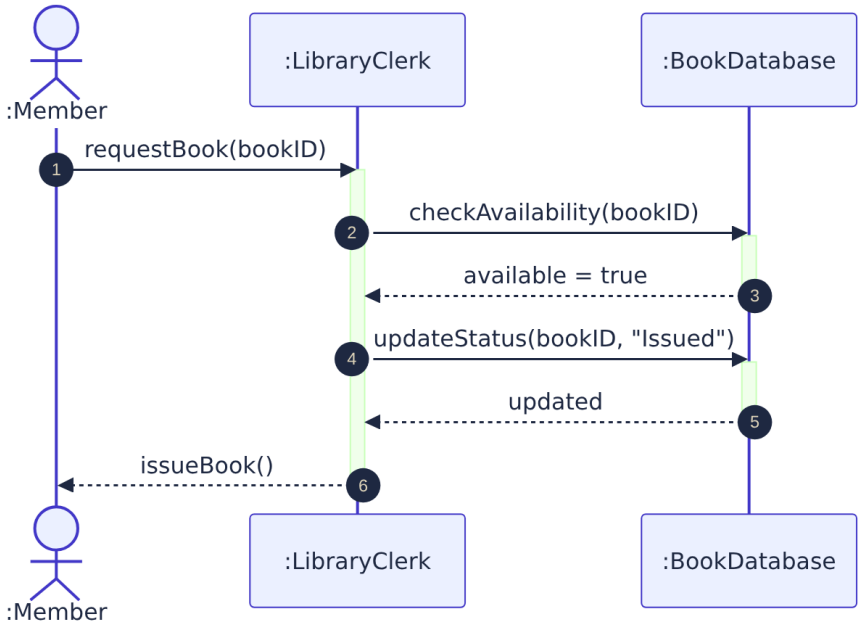
Class diagram	Object diagram
Shows classes (general design)	Shows objects (a snapshot with values)
Name written as *ClassName*	Name written as obj : ClassName
Has attributes and operations	Has attribute values only
Multiplicity is shown	Multiplicity is not shown (only links)

4.5 Interaction diagrams: Sequence and Collaboration

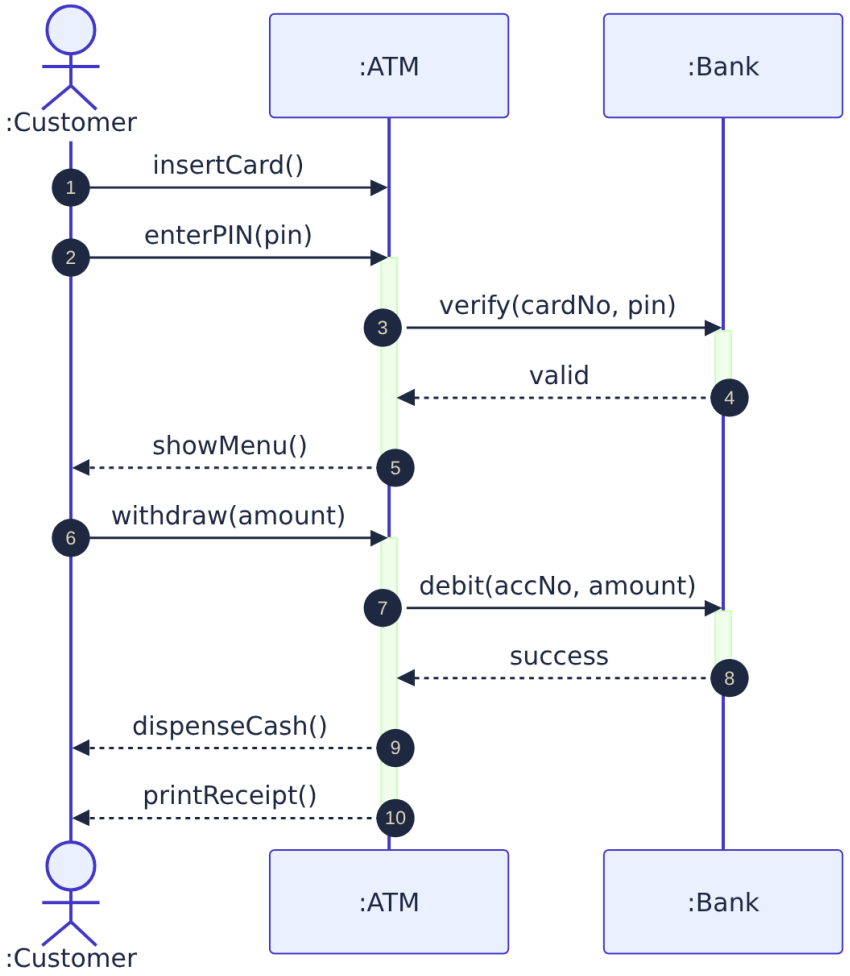
Objects work together by **sending messages**. Interaction diagrams show this. They show **dynamic behaviour**. There are two forms: **Sequence diagram** (time order) and **Collaboration diagram** (object layout).

> **SEQUENCE DIAGRAM**
> Shows **how objects interact, in time order**. **Horizontal axis:** the objects. **Vertical axis:** time, flowing downwards.

Part	Notation
Object	Box at the top with name : Class
Lifeline	Vertical dashed line below the object: shows that the object exists
Activation	Thin vertical rectangle on the lifeline: the object is active / doing work
Message	Horizontal solid arrow from one lifeline to another, labelled with name and arguments (e.g. *checkAvailability(bookID)*)
Return message	Dashed arrow going back
Self message	Arrow that starts and ends on the same lifeline (object calls itself)



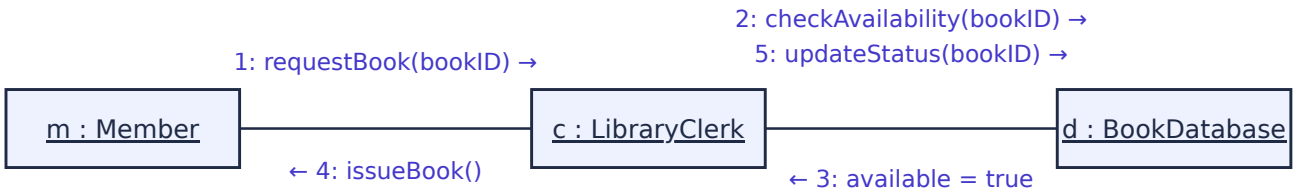
Sequence diagram: Issue book (Library Management System)



Sequence diagram: ATM cash withdrawal

COLLABORATION DIAGRAM

Also shows object interaction, but **focuses on the structure (links between objects)** instead of time. Messages are drawn along the links and are **numbered (1, 2, 3...)** to show the order. It shows the context in which the interaction happens.



Collaboration diagram: Issue book — same interaction as the sequence diagram, drawn with numbered messages

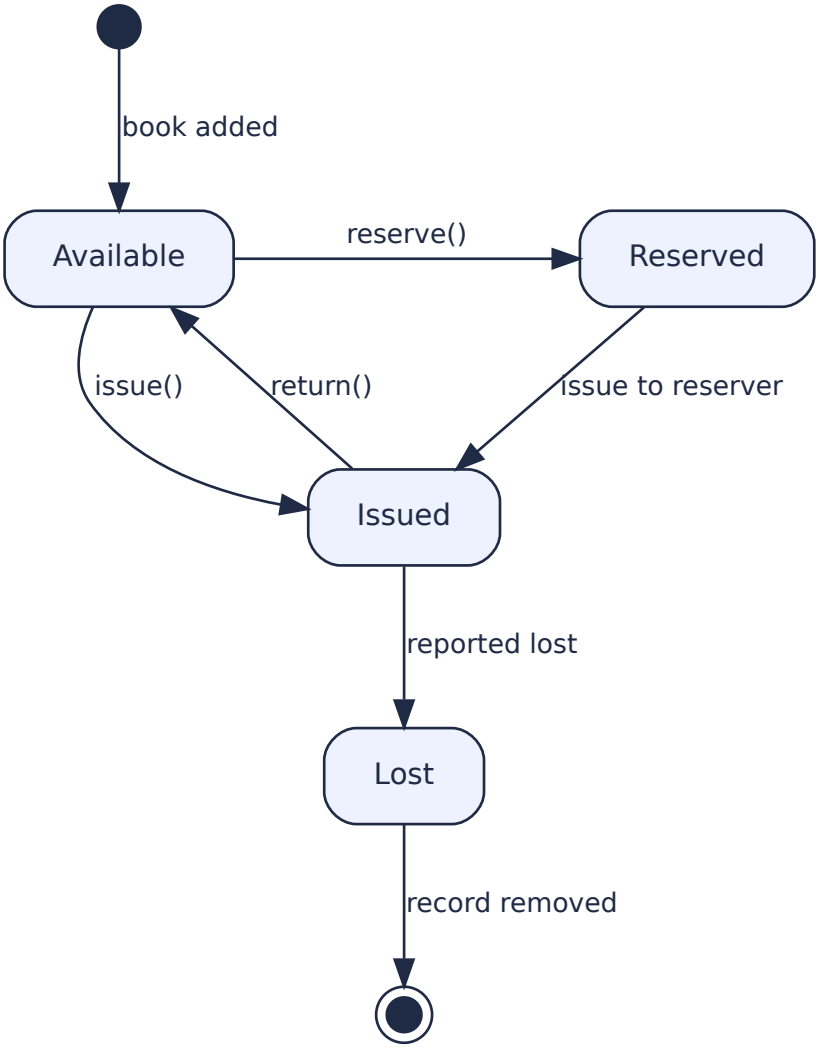
Sequence diagram	Collaboration diagram
Emphasises **time order**	Emphasises **object links / organisation**
Uses lifelines and activation bars	Uses links and numbered messages
Easy to read the order of messages	Easy to see which objects are connected
Time is a dimension (top to bottom)	Time is shown only by sequence numbers
Not good for many objects (gets wide)	Gets crowded with many messages

Both diagrams show the same information, so one can be converted into the other.

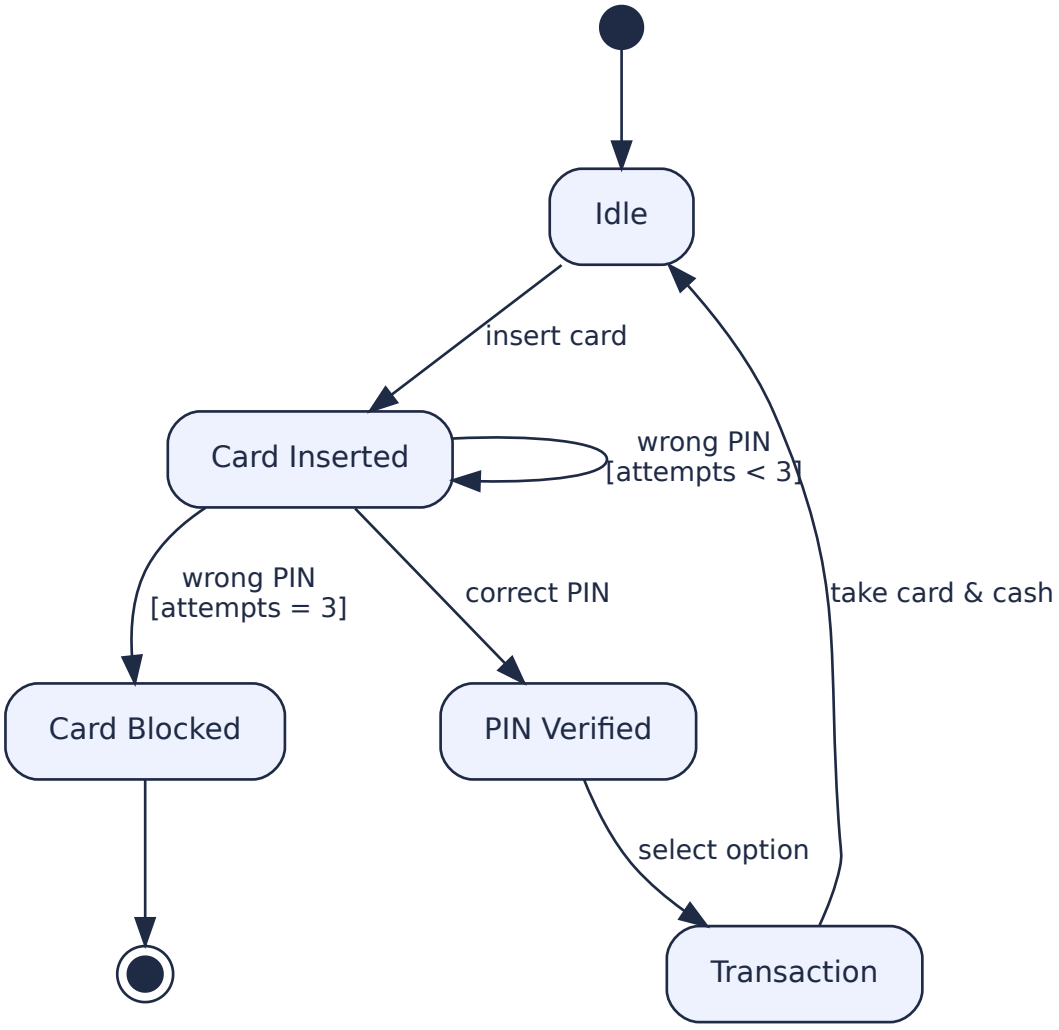
4.6 State-Chart and Activity diagrams

> **STATE-CHART DIAGRAM**
> Shows the **different states of one object** during its life and how it moves from one state to another when an **event** occurs. Used to model the **behaviour of a single object**.

- **State:** rounded rectangle with the state name.
- **Initial state:** small filled circle. **Final state:** filled circle inside a ring.
- **Transition:** arrow between states, labelled `event [guard] / action`.



State-chart: a Book in the library (Available → Issued → Available, Reserved, Lost)

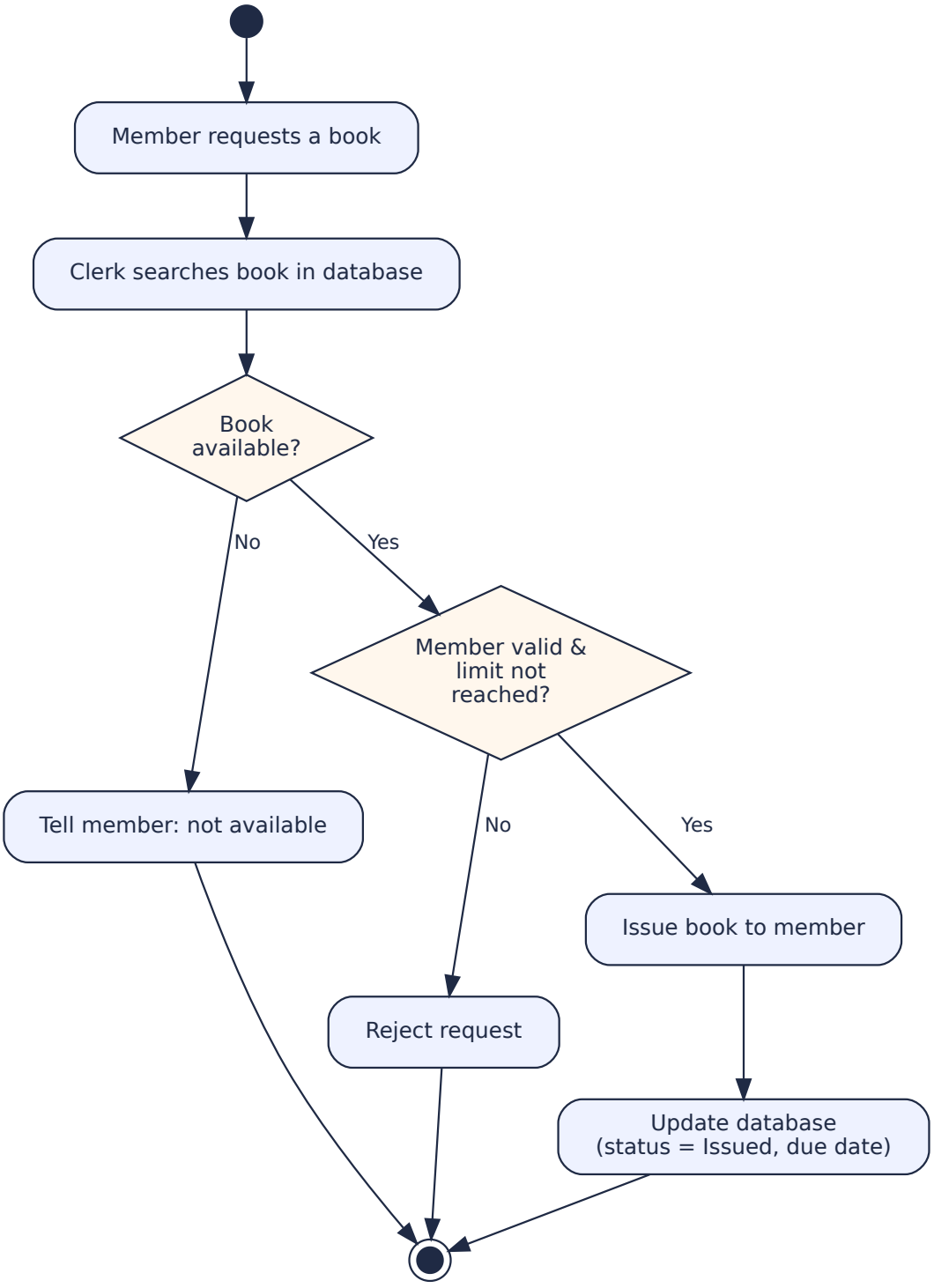


State-chart: ATM card session

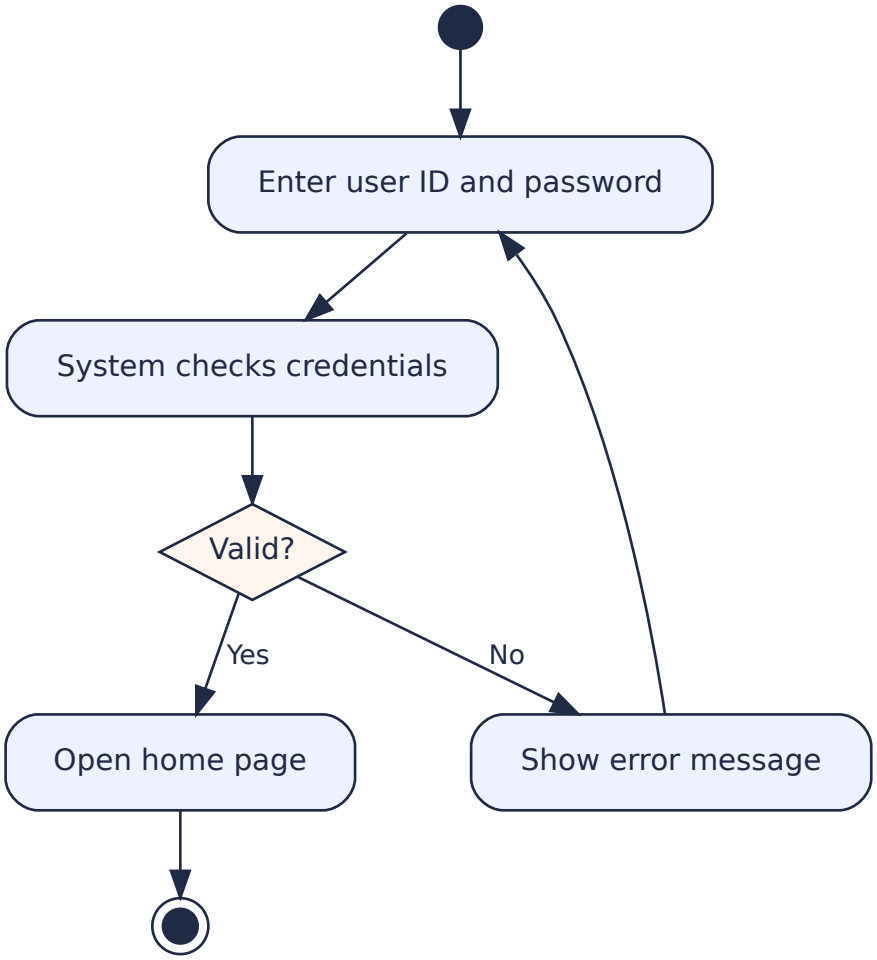
ACTIVITY DIAGRAM

Shows the **flow of control from one activity to another**, like a flowchart for a business process or a use case. It is a special kind of state chart. It shows the **dynamic view** of the system.

- **Action / activity:** rounded rectangle. **Arrow:** completion of an action, moves to the next.
- **Decision (branch):** small diamond with outgoing arrows labelled by **guard conditions** [yes] / [no].
- Start (filled circle) and end (circle in ring). **Fork/join** bars for parallel activities. **Swimlanes** show who does which activity.



Activity diagram: Issue book



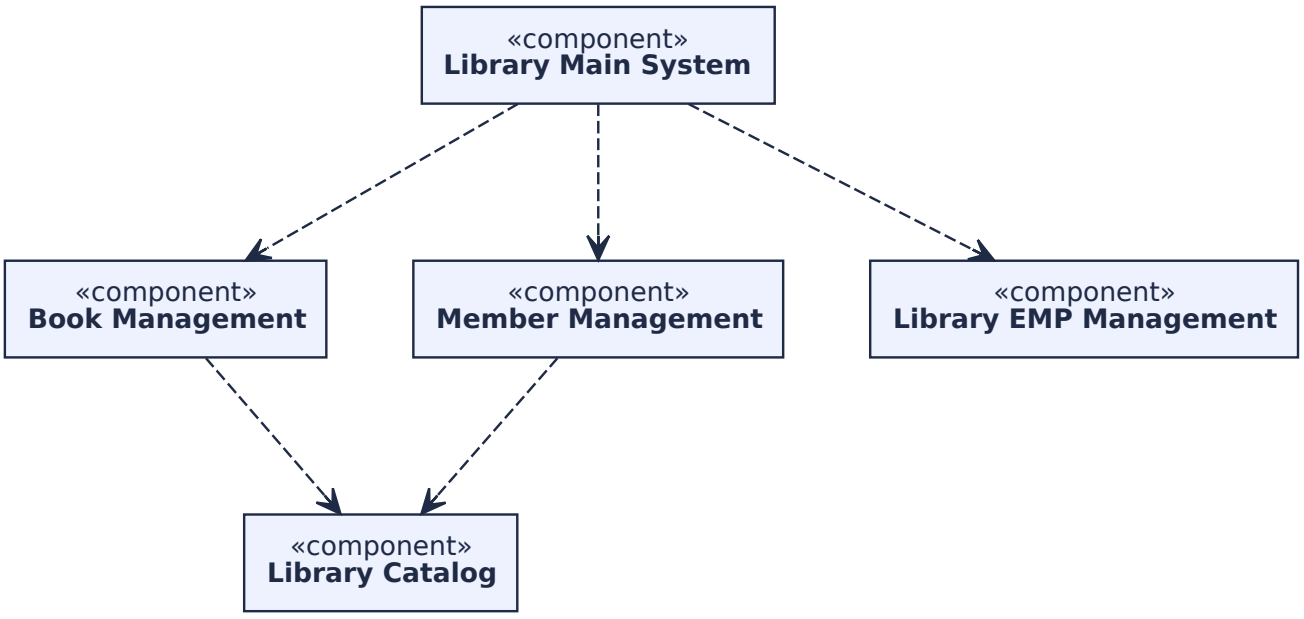
Activity diagram: User login

State-chart diagram	Activity diagram
Focus: **states** of one object	Focus: **activities** / steps of a process
Changes triggered by **events**	Next step starts when the previous activity **completes**
Models the life of one object	Models the workflow across many objects
Example: Book: Available → Issued	Example: steps to issue a book

4.7 Component and Deployment diagrams

COMPONENT DIAGRAM

Shows the **implementation (physical) view**: the software **components** (source code, executables, DLLs, business components) and the **dependencies** between them. Component symbol: rectangle with two small rectangles on the side. Dependencies are dashed arrows.



Component diagram: Library Management — five components (Library Main System, Book Management, Member Management, Library EMP Management, Library Catalog)

DEPLOYMENT DIAGRAM

Shows the **hardware configuration at run time**: **nodes** (computers, servers, devices, drawn as 3-D boxes) connected by communication lines, and which software components **run on** which node. Used to plan installation at the customer site.



Deployment diagram: Library Management — Client desktop, Application server, Database server

Component diagram	Deployment diagram
Software parts and their dependencies	Hardware nodes and where software runs
Implementation view	Deployment (environment) view
Elements: components, interfaces	Elements: nodes, connections, artifacts

4.8 Generalization, Domain Model, Architecture, Packaging and Large-scale systems

Generalizations

Generalization means finding what is **common** in several classes and moving it into a **parent (superclass)**; the specific parts stay in **child classes (subclasses)**. It is an **"is-a"** relation. In UML it is an arrow with a **hollow triangle** pointing to the parent. **Specialization** is the opposite direction (parent → more specific children).

EXAMPLE

Telephone system: *Phone* is the parent; *Mobile* and *Landline* are children. Common: number, call(). Special: Mobile has SIM, Landline has line type. Hospital: Doctor and Patient generalize to Person.

Domain model and its refinement

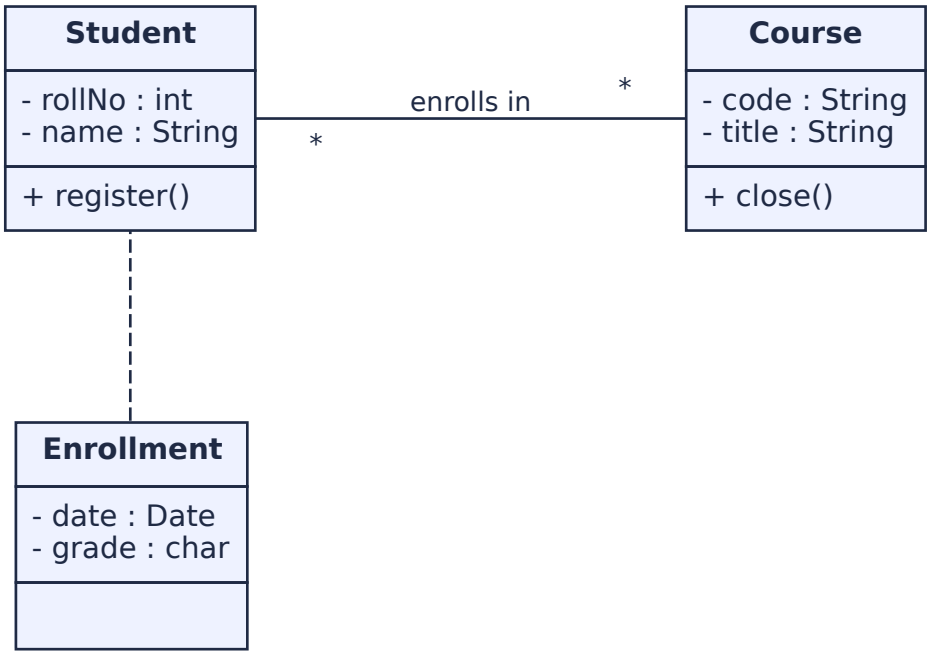
DOMAIN MODEL

A domain model is a **conceptual class diagram of real-world things** in the problem area (business concepts, not software classes). It shows concepts, attributes and relationships. It is built during analysis.

How to build it: read the problem statement; take **nouns as candidate classes**; remove duplicates, vague words and things outside the system; add **attributes** and **associations** with multiplicity.

Refinement means improving the first domain model step by step:

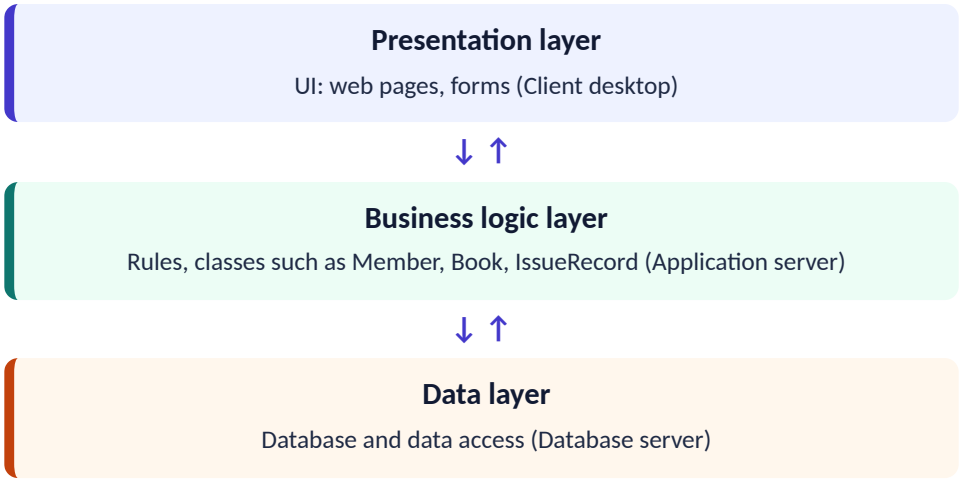
- Add **missing classes** and remove unnecessary ones.
- Replace general associations with **aggregation or composition** where there is a whole-part relation.
- Use **generalization** to remove duplicate attributes (move them to a parent class).
- Add **association classes** for many-to-many links (e.g. Enrollment between Student and Course).
- Add **multiplicity, role names and constraints**.
- Add **operations** and visibility, turning the domain model into a design class diagram.



Refinement example: many-to-many Student–Course refined with the association class Enrollment (date, grade)

Architecture

Software architecture is the overall structure of the system: the major parts (subsystems), their responsibilities, and how they communicate. In UML, architecture is described with the **4+1 views**: use case, class (logical), component, deployment, and process views. A very common style is **layered (3-tier) architecture**:

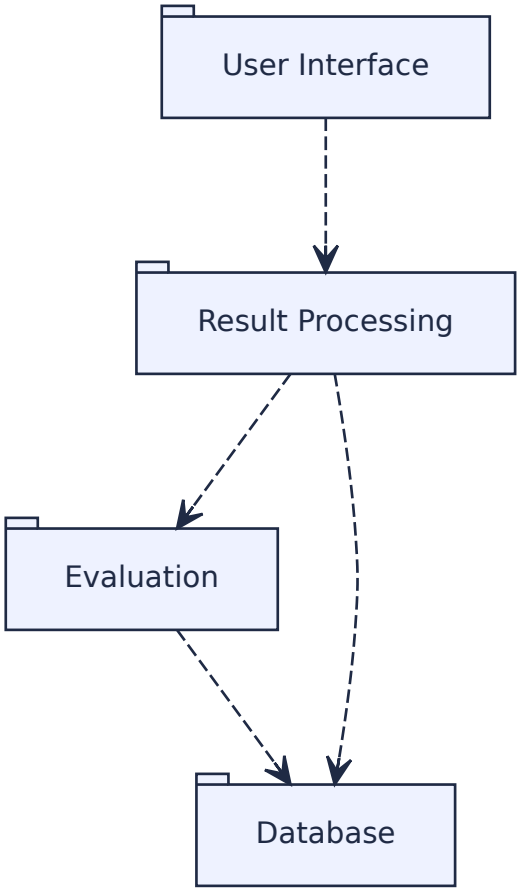


Advantages: each layer can be changed or tested separately, easy reuse and maintenance, clear separation of concerns.

Packaging model elements (package diagram)

PACKAGE

A **package** is a container that **groups related model elements** (classes, components, use cases, even other packages). It gives a **modular view** of a large system. Drawn as a **folder (rectangle with a tab)**. **Dependencies** between packages are dashed arrows.



Package diagram — Result Processing depends on Evaluation (example from slides)

Rules for good packaging: group classes that change together; keep packages **highly cohesive and loosely coupled**; avoid circular dependencies; give each package a clear name.

Case study of a large-scale system — Online Examination / E-commerce

Large systems are too big to draw in one diagram, so we **divide them into subsystems (packages)** and use a different diagram for each purpose:

Stage	UML work done (example: Online Shopping System)
Requirements	Use case diagram: Customer, Admin, Payment Gateway (see 4.9)
Domain model	Customer, Product, Cart, Order, Payment, Category
Packaging	Packages: User management, Product catalog, Order processing, Payment
Dynamic behaviour	Sequence diagram for "place order"; state chart for Order (Placed → Paid → Shipped → Delivered); activity diagram for checkout
Architecture	3-tier: web browser → application server → database server
Implementation	Component diagram (modules) and deployment diagram (servers)

Another case from your slides: **EtransQ** (suppliers, agents, transporters) with use cases like registration, tendering, offering, testimonial.

4.9 Probable UML diagram questions (with solutions)

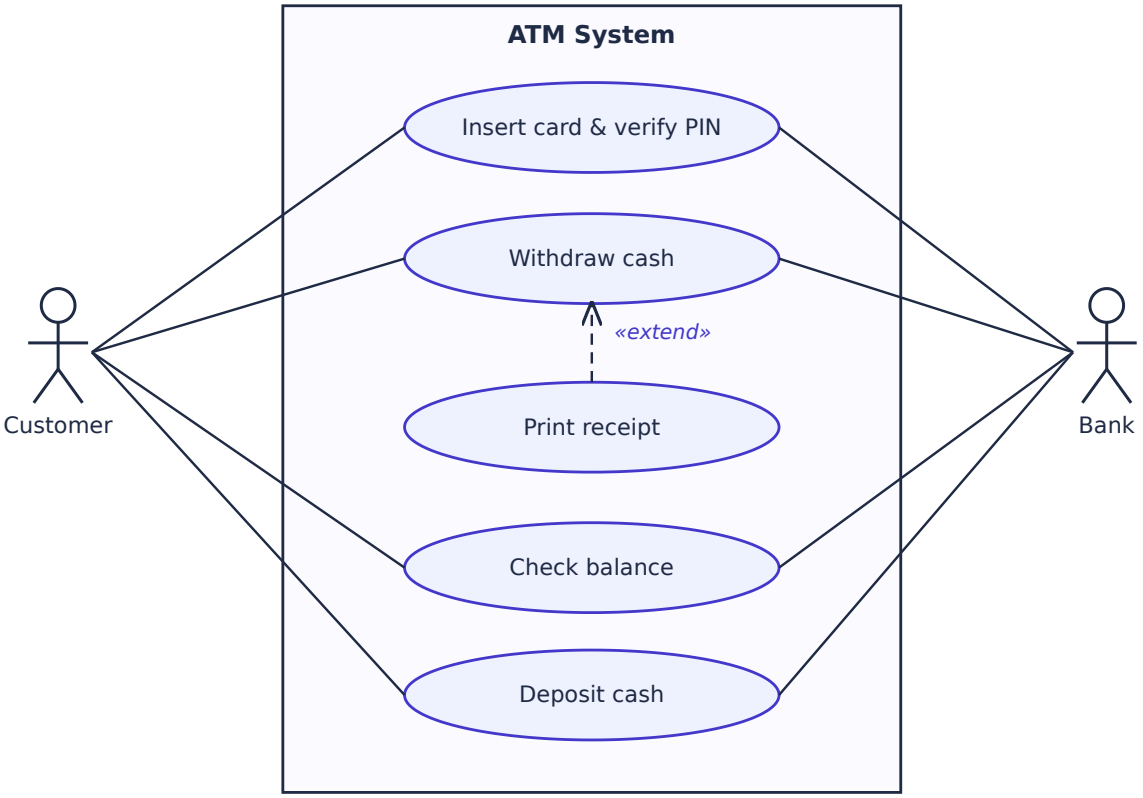
HOW TO ANSWER ANY "DRAW A ... DIAGRAM FOR ..." QUESTION

1) Write one line: what the system is. 2) List actors / classes / objects you will use. 3) Draw a neat diagram with proper symbols. 4) Write 2–3 lines explaining it. Always label arrows and multiplicity.

Q1. DRAW THE USE CASE DIAGRAM FOR A LIBRARY MANAGEMENT SYSTEM AND WRITE THE SCENARIO OF ISSUE BOOK.

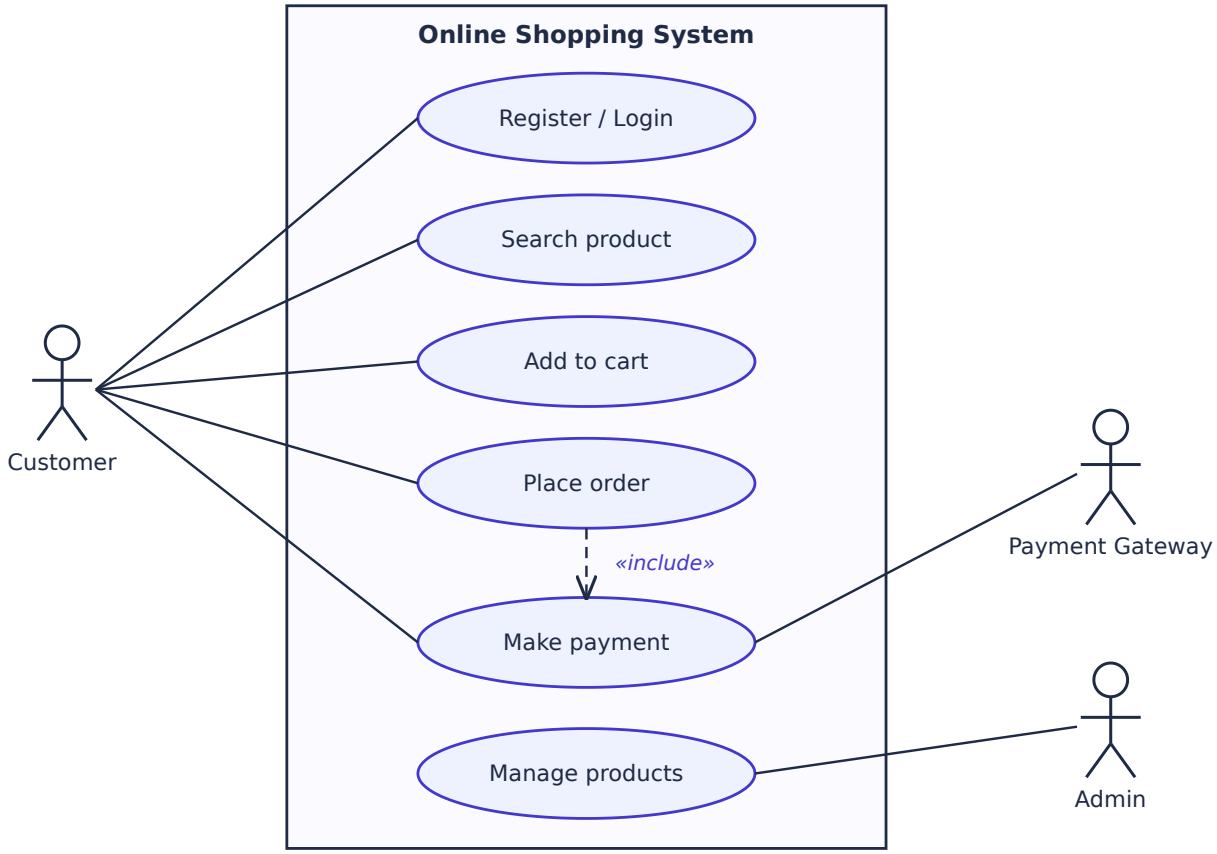
Answer: Use the diagram of 4.3 (actors Member, Librarian, Library clerk; 7 use cases) and the table of UC-05 (Issue books).

Q2. DRAW THE USE CASE DIAGRAM FOR AN ATM SYSTEM.



ATM use case diagram: Customer withdraws, checks balance, deposits; Bank verifies; "Print receipt" extends "Withdraw cash"

Q3. DRAW THE USE CASE DIAGRAM FOR AN ONLINE SHOPPING SYSTEM.



Online shopping: Customer, Payment Gateway and Admin; "Place order" includes "Make payment"

Q4. DRAW THE CLASS DIAGRAM FOR ATM / LIBRARY / HOSPITAL SYSTEM.

Answer: See the three class diagrams in 4.4. In the exam, draw only 4–6 classes with 2–3 attributes and operations each, and show multiplicity and one generalization.

Q5. DRAW THE SEQUENCE DIAGRAM FOR ISSUING A BOOK / ATM WITHDRAWAL.

Answer: See the two sequence diagrams in 4.5. Remember: actor on the left, lifelines, activation bars, numbered messages, dashed returns.

Q6. DRAW THE STATE-CHART DIAGRAM OF A BOOK / ATM / ORDER / STUDENT ACCOUNT.

Answer: See 4.6. For an **Online order** use: Placed → Paid → Shipped → Delivered, with Cancelled from Placed or Paid.

Q7. DRAW THE ACTIVITY DIAGRAM FOR ISSUE BOOK / LOGIN / ONLINE REGISTRATION.

Answer: See 4.6. Always start with the filled circle, use diamonds for decisions with [Yes]/[No], and end with the final node.

Q8. DRAW COMPONENT AND DEPLOYMENT DIAGRAMS FOR THE LIBRARY MANAGEMENT SYSTEM.

Answer: See 4.7 (5 components; 3 nodes: client desktop, application server, database server).

Q9. DRAW THE DFD (LEVEL 0, 1, 2) OF THE EXAMINATION SYSTEM.
Answer: See 3.3, mam's solved example.

Q10. DRAW THE COLLABORATION DIAGRAM FOR ISSUE BOOK.
Answer: See 4.5. Same messages as the sequence diagram but drawn on links with numbers 1, 2, 3 ...

Practice: Student Result (Examination) System — try it yourself

QUESTION
Draw a use case diagram and a class diagram for a Student Examination System.

HINTS: USE CASE
Actors: Student, Professor, Registrar (Exam Committee).
Use cases: Fill exam form, Pay fees, Set question paper, Attend exam, Evaluate answers («include» Apply negative marking), Generate result, Generate merit list.

HINTS: CLASS DIAGRAM
Classes: Student(rollNo, name), Subject(code, name), Exam(date), Question(text, answer), Result(marks, grade), MeritList(class, rank).
Links: Student 1—* Result; Exam 1—* Question (composition); Result belongs to Subject.

4.10 Quick revision

All nine UML diagrams on one page

Diagram	Type	Shows	Key symbols
Use case	Behavioural (user view)	System functions for actors	Stick figure, ellipse
Class	Structural	Classes and relationships	3-part box, diamonds, triangle
Object	Structural	Snapshot of objects with values	Underlined name
Sequence	Dynamic	Messages in time order	Lifeline, activation, arrows
Collaboration	Dynamic	Messages between linked objects	Links, numbered messages
State chart	Dynamic	States of one object	Rounded box, transitions
Activity	Dynamic	Flow of activities	Rounded box, diamond, bars
Component	Implementation	Software parts, dependencies	Component box, dashed arrows
Deployment	Implementation	Hardware nodes and software on them	3-D cube

Important differences

Compare	Difference in one line
Aggregation vs Composition	Hollow diamond: part can exist alone. Filled diamond: part dies with the whole.
Include vs Extend	Include is always executed; extend is optional / conditional.
Class vs Object diagram	Class diagram = design; object diagram = snapshot with values.
Sequence vs Collaboration	Time order vs object links; both show the same interaction.
State chart vs Activity	States of one object vs flow of activities in a process.
Component vs Deployment	Software structure vs hardware setup.
Association vs Dependency	Association is a lasting link (solid line); dependency is a temporary use (dashed arrow).
Generalization vs Realization	Generalization: class inherits class (solid line, hollow triangle). Realization: class implements interface (dashed line, hollow triangle).

Symbols cheat sheet

Item	Symbol	Item	Symbol
Association	—— plain line	Dependency	- - → dashed arrow
Aggregation	◇—— hollow diamond	Realization	- - ▷ dashed + hollow triangle
Composition	◆—— filled diamond	Generalization	—▷ solid + hollow triangle
Public / Private / Protected	+ - #	Stereotype	« » guillemets

LAST-MINUTE CHECKLIST

☐ Learn the Examination System DFD (levels 0, 1, 2). ☐ Practise one use case, one class and one sequence diagram by hand. ☐ Remember differences (include/extend, aggregation/composition). ☐ Write definitions in 1–2 lines, then points, then diagram. ☐ Label every arrow.

All the best for Test-2!